Overlapping communities as connected components of a commonality hypergraph

Dmitri Deinega,[1] Igor Kanovsky,[2] and Baruch Barzel[1]

[1] *Department of Mathematics, Bar-Ilan University, Ramat Gan 5290002, Israel*
[2] *Max Stern Yezreel Valley College, Emek Yezreel, Israel*

Nodes that share most of their neighbours tend to share a community. On this premise, three nodes, one adjacent to the other two, whose three pairs all reach a commonality threshold form an elementary community, and communities are the connected components of the resulting 3-uniform hypergraph under shared pairs. The construction, HyperCommon, is deterministic, local and nested in its threshold, and no pair of nodes lies in elementary communities of two different communities. We derive the exact window of thresholds that recovers planted overlapping ring lattices whose shared nodes lie far apart. We compare HyperCommon with thirteen other methods on twenty-eight ring-lattice settings that vary community size, density, overlap and noise model. With every parameter chosen without labels by overlapping modularity (Newman–Girvan modularity for partitions), HyperCommon has the highest mean Omega index in sixteen of them, most clearly where communities of unequal size and density overlap, and is level with the best other method in five; it trails mainly on dense rings with little overlap and on a gradient of ring sizes. With each method's best parameter it leads in eleven and trails in thirteen, and with one parameter frozen per family it leads in one. It trails on all eight LFR families, whose communities are random graphs internally, and once a tenth (with labels) or three tenths (without) of each ring's edges are rewired internally, while the communities are still clustered. On seventeen real networks of up to 1.1×10^6 nodes, the threshold chosen without labels keeps a median 94% of the best threshold's F1, but on most of them another method is more accurate.

I. INTRODUCTION

Most real networks are organised into groups of nodes that are more densely connected to one another than to the rest of the network [1–4]. In social, biological and information networks these groups overlap: a person belongs to a family, a workplace and a sports club, and a protein to several complexes. Methods that partition the nodes cannot represent this, and the many methods designed for overlapping communities [5, 6] differ less in how they search than in what they take a community to be. Modularity optimisation [7–9] compares a group with a global null model, flow-based methods [10] with a random walker's trajectories, statistical inference [11–14] with a generative model, OSLOM [15] with the significance of each group against a configuration-model null, label propagation [16–18] with a consensus among neighbours, and local methods [19, 20] with a fitness grown from a seed. No definition is best on every network, in theory [21] or across real data [22]; the useful question about a method is which structure it captures, and why.

A natural definition comes from the inside: two friends who share most of their friends are, more likely than not, in the same circle. This is triadic closure [23, 24]. Real networks are highly clustered [25], so neighbourhoods overlap, and the overlap is larger between members of a group than across groups. Since triadic closure and community structure both produce shared neighbourhoods, and separating the two is itself an inference problem [26], Sec. III tests the premise on networks with recorded groups. We call the overlap of two closed neighbourhoods the *commonality* of the pair. Pairwise neighbourhood similarity is an old tool – Jaccard's coefficient [27], vertex similarity [28], the local indices of link prediction [29, 30] and their global corrections for indirect similarity [31, 32], SCAN [33], link communities [34, 35], and clique and k-truss communities [36–40] – but a single pair is a weak and degree-confounded signal; what matters is how pairwise evidence is combined.

We combine it through triples. Three nodes of which one is adjacent to the other two, and whose three pairs all have commonality at least t, form an elementary community; two such triples belong to the same community when they share a pair. The communities are therefore the connected components of the two-node-overlap graph of a 3-uniform hypergraph, which gives the method its name, HyperCommon. Higher-order structures are increasingly seen as natural units of network organisation [41–44]; here the hypergraph is built from pairwise commonality rather than given. A node shared by two communities belongs to triples in both, so overlap needs no extra machinery. The construction differs from 3-clique percolation [36, 45] in two ways that we test separately: open wedges (three nodes joined by exactly two edges) are admitted, as in triad percolation [46], so communities need not be dense in triangles, and every pair of a triple – including the pair that is not an edge, which link communities test alone [34] – must pass the commonality test.

This paper makes four contributions.

(i) *The premise, tested.* On four networks with recorded memberships, edges whose endpoints share a recorded group have higher commonality than edges whose labelled endpoints share none – on the three large ones at every endpoint degree – once unlabelled nodes are no longer counted as evidence of separation (counting them reverses the conclusion on DBLP). Every member of a family

of neighbourhood-overlap functions carries the signal, and the most informative one depends on the network (Sec. III).

(ii) *A construction with exact guarantees.* HyperCommon combines the non-adjacent-pair test of link communities with the edge tests of thresholded clique percolation and SCAN. Like any threshold percolation of locally tested triples, its cover is independent of processing order, computable locally from any seed triple and nested in the threshold; no pair of nodes lies in admitted triples of two different communities, so every edge of an admitted triple is assigned to exactly one community. On overlapping ring lattices with isolated shared nodes we derive the exact window of thresholds that recovers the planted communities, also for unequal densities, and an exact implementation confirms it (Secs. IV, V and VII A).

(iii) *A benchmark under four protocols.* We compare HyperCommon with three ablations and thirteen methods on the benchmarks (twelve on the fourteen smaller real networks, fewer on the three largest), among them link communities and Leiden with the Constant Potts Model, on ring lattices, overlapping LFR graphs [47, 48] and seventeen real networks (Secs. VI and VII); without the test of the non-adjacent pair, accuracy falls in all 36 benchmark settings, with and without labels. With every parameter chosen without labels by overlapping modularity, which for partitions is the Newman–Girvan modularity, HyperCommon has the highest mean Omega index on 16 of the 28 ring settings and is level on 5 more; it trails on dense rings with little overlap and on one setting of rings with unequal degrees, where the Leiden objectives lead, and on the gradient of ring sizes, where link communities lead. At each method's own default or selection rule it leads on 17 of the 28, at each method's best parameter on 11, and with one parameter frozen per family on 1. It trails on all eight LFR families, and on the rings once a tenth (with labels) or three tenths (without) of each community's edges are rewired internally, long before the communities become random graphs. Controls (degree-preserving rewiring, split overlap, random geometric communities) tie this to the lattice-like geometry of the communities rather than to changes of degree or to the way they share nodes; with hubs inside the lattice communities HyperCommon's own accuracy changes by at most 0.03, but Leiden then leads on the dense rings and the Constant Potts Model on two worlds, with and without labels. On clustered graphs without planted communities HyperCommon, like every method tested, reports many.

(iv) *Scale and parameter choice.* The wedges are scored once and every threshold is read off the scores: scoring the wedges and reading off the covers at all 30 thresholds takes about a minute on one core on Amazon and DBLP (3×10^5 nodes), against 22–27 minutes for Leiden's sweep of resolutions (the overlapping modularity that the label-free choice needs adds 9.0 and 7.7 minutes in our Python implementation), and on YouTube (1.1×10^6 nodes) every threshold of the grid, with its overlapping modularity, is available in ten minutes on a workstation with 28 logical processors (Secs. V E and VII G). On seventeen real networks the label-free threshold retains a median 94% of the best threshold's F1, although other methods are more accurate on most of them, and the best commonality function differs between networks (Sec. VII F).

II. RELATED WORK

Neighbourhood overlap. That adjacent nodes sharing many neighbours tend to share a community underlies the edge clustering coefficient of Radicchi *et al.* [49], the structural similarity of SCAN [33] and the shared-near-neighbour clustering of Jarvis and Patrick [50], and neighbourhood overlap tracks tie strength in social networks [51]. The same indices – common neighbours, Jaccard, Salton's cosine, resource allocation – are the local predictors of link prediction [29, 30], where no single index is best on every network [52]. Jarvis–Patrick clustering and TECTONIC [53], which weights each edge by its triangle count divided by the sum of its endpoint degrees, threshold scored edges and take connected components, while SCAN grows clusters from core vertices with many similar neighbours; Chen and Tsourakakis [54] base the edge weight on triangle and wedge counts to prevent TECTONIC's over-segmentation. Our two-test ablation is of this kind, restricted to wedges.

Clique and truss percolation. Clique percolation [36] joins k-cliques sharing $k-1$ nodes, and Palla *et al.* already applied it to weighted networks after removing links below a weight threshold; the sequential algorithm of Kumpula *et al.* [45] obtains the communities for every threshold in one sweep, and CPMw [38] thresholds a clique's intensity, the geometric mean of its link weights, instead. For $k = 3$, triangles connected through shared edges are also the truss communities [39, 40], which nucleus decompositions organise into a hierarchy [55]. Our triangles-only ablation is 3-clique percolation on the graph thresholded at $J \geq t$ (closed-neighbourhood Jaccard), which we call J-thresholded 3-clique percolation (not the intensity-based CPMw). Triad percolation [46], motivated by the scarcity of k-cliques in sparse online social networks, already admits open triads, expanding seeds until a stopping threshold is met; HyperCommon instead admits a wedge only when all three of its pairs pass a commonality threshold and returns the connected components of all admitted triples, without seeds or a stopping rule.

Link communities. Ahn, Bagrow and Lehmann [34]

cluster edges: two edges $\{c, i\}$ and $\{c, j\}$ are similar when the closed neighbourhoods of i and j overlap, with similarity $J(i, j)$, and single-linkage clustering, cut where the partition density is largest, gives overlapping node communities; hyperlink communities extend this to higher-order networks [56]. In our language, $J(i, j)$ is the commonality of the third pair of the triple $\{i, c, j\}$, the pair that is not an edge when the wedge is open. Line graphs and clique graphs [35, 57] frame such edge- and clique-level clusterings, and the two-node-overlap graph L_t of our construction is the 2-line graph of the hypergraph $\mathcal{H}_t$ [58].

What HyperCommon combines. HyperCommon tests all three pairs of every wedge: the two edges, as thresholded clique percolation and SCAN do, and the non-adjacent pair, as link communities do. It then connects the admitted triples through shared pairs, including non-adjacent ones; this changes the accuracy little on rings when the parameter is chosen per graph and lowers it slightly on overlapping LFR graphs (Sec. VII C), and we keep it because then all admitted triples through any pair, not only through an edge, lie in one community (Corollary 1). Each ingredient is tested separately in Sec. VII, with link communities cut at every level of our threshold grid and a variant joining triples through shared edges only beside the two ablations above. Determinism and nesting in the threshold are shared with thresholded clique percolation [45] and single-linkage link communities [34], and local queries of the same kind are answered for k-truss communities [40]; what HyperCommon adds is the exact-recovery window (Theorem 2), which ties the threshold to the structure it recovers.

Resolution and threshold choice. Modularity has a resolution limit [59]; the absolute Potts model of Ronhovde and Nussinov [60] and the Constant Potts Model [61], equivalent to it on unweighted graphs, avoid it while keeping one global parameter. To choose the threshold without labels we use the overlapping modularity of Shen *et al.* [62], introduced to select a level of a hierarchy of overlapping communities; overlapping modularity measures peak close to the critical point of k-clique percolation [63]. We apply the same rule to every method; for a partition it is the Newman–Girvan modularity, so we also score every method at its own default or selection rule (Sec. VI). Besides Leiden with modularity and the Constant Potts Model, we compare with label propagation [16], SLPA [18], the ego-network methods DEMON and ANGEL [64, 65], Walktrap [66], Infomap [10], whose Markov time sets its scale [67, 68], and, among statistical-inference methods [11–14], the nested stochastic block model [11], which selects its own number of groups; label propagation, Walktrap and Infomap return partitions. Overlapping inference (BigClam [14]), OSLOM [15] and ego-splitting [69] are not among the methods compared (Sec. VI). The construction was first formulated, in an order-dependent form, in Ref. [70].

III. COMMONALITY IN EMPIRICAL NETWORKS

The premise of the method is local: two adjacent nodes whose neighbourhoods largely coincide belong, more often than not, to a common community. We test it on four networks with recorded group memberships (Table VI): Zachary's karate club [71], with its two post-split factions, and the DBLP co-authorship, Amazon co-purchasing and YouTube friendship networks, whose ground-truth groups (publication venues, product categories and user-created groups) come from the Stanford Network Analysis Project (SNAP) [72].

A. Unlabelled nodes and degree

The SNAP networks label only part of their nodes: 82% in DBLP, 95% in Amazon and 4.6% in YouTube. Counting pairs with an unlabelled node as pairs in different groups, as the first author's earlier analysis did [70], is the standard pitfall of learning from positive and unlabelled data [73]; on DBLP it reverses the conclusion, the area under the ROC curve (AUC) falling to 0.41, below chance. We therefore classify every edge as *inner* (both endpoints labelled and sharing a group), *outer* (both labelled, sharing none) or *unlabelled*, and compare inner with outer only. "Outer" means no *recorded* shared group, and metadata need not coincide with structural communities [21, 74]: the analysis tests a tendency, not a classifier. With this correction commonality separates the classes on all four networks, with AUCs of 0.82, 0.62, 0.60 and 0.61 in the order above (Supplemental Material [75], Fig. S2).

Jaccard commonality is capped by the degree ratio of the pair, $J(u, v) \leq (d_{\min} + 1)/(d_{\max} + 1)$, so edges incident to hubs have low J whatever their community structure, while degree also correlates with how labels were assigned. Within strata of the larger endpoint degree the separation persists at every degree [Fig. 1(a)]: the stratum AUCs range from 0.75 to 0.87 in DBLP, all above the pooled 0.62, and lie between 0.58 and 0.67 in Amazon and between 0.53 and 0.67 in YouTube.

B. Commonality is a family of functions

Jaccard is one of many overlap scores catalogued for link prediction [28–30]. With $I = |N[u] \cap N[v]|$ and $U = |N[u] \cup N[v]|$ we compare eight (defined with Fig. S3): I, Jaccard I/U, I^2/U, $I/U^{1/2}$, the cosine [76], the hub-promoted $I/\min$ (after Ravasz *et al.* [77]) and hub-depressed $I/\max$, and resource allocation (RA). Within strata of both endpoint degrees every one separates inner from outer edges on the three SNAP networks [Fig. 1(b)], and the overlap-based scores perform alike (DBLP AUC 0.81–0.85, Amazon 0.60–0.62, YouTube 0.60–0.63): at fixed endpoint degrees each is a monotone function of

I. They differ on pairs of *different* degrees, which a single global threshold must compare. Pooled over all edges, no function is best everywhere: I leads on DBLP (0.78, against 0.62 for Jaccard) but falls below chance on YouTube (0.47), where hubs dominate, and $I/\min$ leads on the other three networks (0.89, 0.62, 0.69). HyperCommon takes any member, the appropriate one depending on the network (Sec. VIII); unless stated otherwise we use Jaccard, the similarity of link communities [34].

C. From pairs to triples

HyperCommon requires all three pairs of a wedge to pass the threshold, including the pair that is not an edge. On unique labelled triples, call a triple *pure* when one recorded group contains all three nodes, and compare the *pair rule*, which tests only the edges of the triple (as in the two-test ablation of Sec. VI), with the *triple rule*, which also tests the non-adjacent pair of an open wedge. At equal recall from 0.1 to 0.4 the triple rule is the more precise on all three SNAP networks, with 95% intervals of the difference that exclude zero [Fig. 1(c)]; at recall 0.2, 81% of pair-admitted triples are pure against 93% of triple-admitted ones in DBLP. Beyond recall ≈ 0.5, where most wedges are admitted, the pair rule is the more precise on DBLP (Fig. S4). Section VII tests the third test and the admission of open wedges on the final communities.

IV. THE METHOD

HyperCommon has two settings: the commonality function f, which scores a pair of nodes (closed-neighbourhood Jaccard unless stated otherwise), and the threshold t. Given a graph G:

1. For every wedge $\{i, j, \ell\}$ – three nodes, one adjacent to the other two, each counted once whether open or closed – compute f on its three pairs.

2. Admit the wedge as a hyperedge if all three values are at least t.

3. Join two admitted triples when they share two nodes, and return, for every connected component, the union of its triples' nodes.

Commonality values are memoised per pair, and components are found on the bipartite incidence graph between admitted triples and the pairs they contain, which has the same triple components as the two-node-overlap graph and is linear in the number of triples. Because the triple scores $s(T) = \min f$ over the pairs of T do not depend on t, the covers for every threshold follow from a single enumeration, each at the cost of one component search. The threshold can be set from the graph alone, by maximising the overlapping modularity of the cover [62] (Secs. VI D and VII A). A local variant grows the community of a given seed triple by breadth-first search over shared pairs, reading only the neighbourhoods of the nodes it reaches and of the candidates it examines (Proposition 2).

Figure 2 illustrates the construction. Nodes in no admitted triple remain unassigned, and communities have at least three members: the method reports what local evidence supports, and coverage is reported for every method in Tables S12 and S14 of the Supplemental Material [75].

V. PROPERTIES OF THE COMMONALITY HYPERGRAPH

Throughout, $G = (V, E)$ is a finite simple undirected graph, $N[u] = \{u\} \cup N(u)$ is the closed neighbourhood of u, and

$$J(u, v) = \frac{|N[u] \cap N[v]|}{|N[u] \cup N[v]|} \tag{1}$$

is the commonality of the pair. A set $T = \{i, j, \ell\}$ of three distinct nodes is a *wedge* if one of its nodes is adjacent to the other two, and it is *admissible at threshold* t if it is a wedge and $J \geq t$ for all three of its pairs. The admissible triples $\mathcal{T}_t$ are the hyperedges of a 3-uniform hypergraph $\mathcal{H}_t = (V, \mathcal{T}_t)$. Connectivity is taken through shared pairs rather than shared nodes: let L_t be the two-node-overlap graph of $\mathcal{H}_t$, whose vertices are the triples of $\mathcal{T}_t$, two of them adjacent when they share exactly two nodes. The communities at threshold t are the node sets of its components, one community per component:

$$\mathcal{C}_t = \left(\bigcup_{T \in K} T \right)_{K \text{ a connected component of } L_t}. \tag{2}$$

A node belongs to every community whose component contains a triple through it, so $\mathcal{C}_t$ is a cover rather than a partition, and nodes in no admissible triple are left unassigned. Propositions 1, 2 and 4, Theorem 1, Corollary 1 and Proposition 3(i) use only the definition of admissibility and hold for any commonality function in place of J; Proposition 3(ii) holds for any function that is at most one and equals one exactly on identical closed neighbourhoods, as Jaccard, the cosine and the overlap normalised by the larger neighbourhood do; Lemma 2 and Theorem 2 are specific to Jaccard. All proofs are in Appendix C.

A. Determinism and locality

Proposition 1 (Order independence)**.** *$\mathcal{C}_t$ is a function of (G, t) alone: it does not depend on a processing order, a random seed, or the order in which triples are discovered.*

$\mathcal{T}_t$, L_t and its components are all defined pointwise from (G, t). This is the property the procedure of the original formulation [70] lacked: it deleted each edge once used, so whether a later triple could still be formed depended on which community had been grown first.

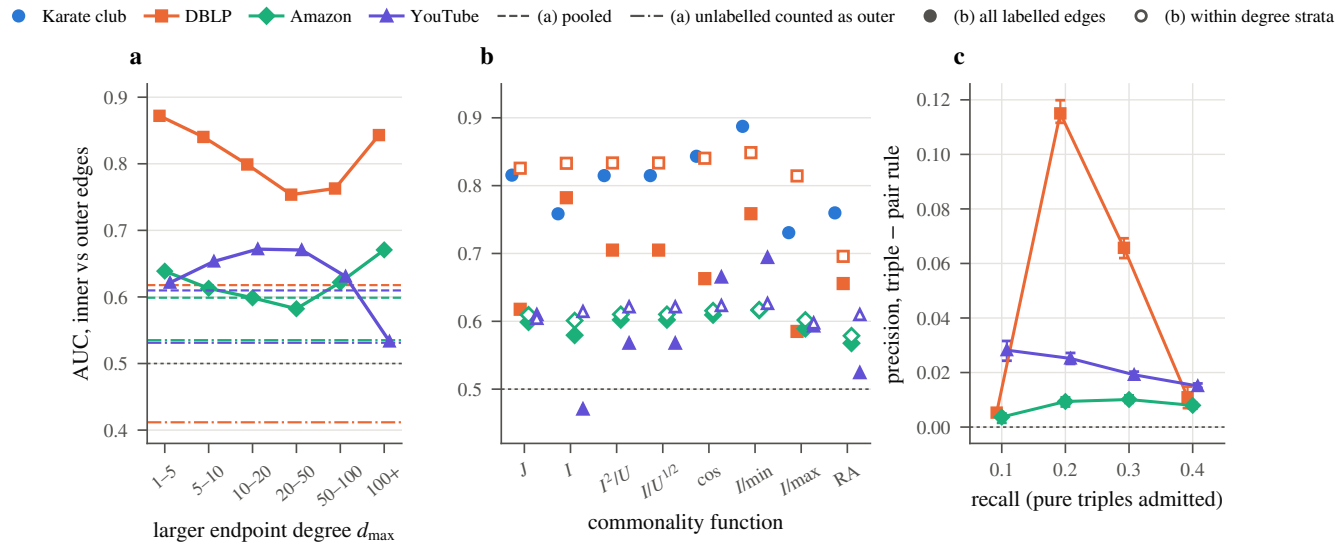


FIG. 1. Commonality in networks with recorded memberships. (a) AUC of closed-neighbourhood Jaccard for inner against outer edges within strata of $d_{\max}$ (points), pooled (dashed), and with unlabelled edges counted as outer (dash-dotted), as in Ref. [70]; the karate club is too small to stratify. (b) AUC of eight functions over all labelled edges (filled) and within strata of both endpoint degrees (open). (c) Precision of the triple minus the pair rule at equal recall, with 95% intervals from resampling the triples.

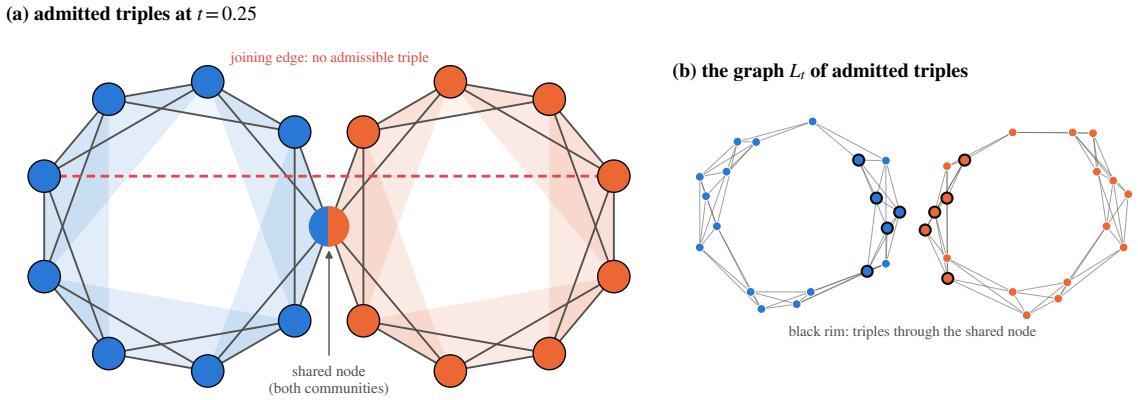


FIG. 2. The method on a toy graph, computed by the implementation. (a) Two ring lattices of nine nodes ($k = 4$) share one node, and one edge, placed by hand for the illustration between two nodes far from the shared node (it closes no triangle with it), joins them. Shaded triangles are the triples admitted at $t = 0.25$, coloured by component; the shared node is admitted in both, the joining edge in none. (b) The graph of admitted triples, linked when they share a pair (L_t), each component drawn on its own circle; its two components are the two communities. The rings have $s = 9 < 2k + 2$, outside the stated hypothesis of Theorem 2 but within the ring sizes for which its window still holds (remark after the theorem).

Proposition 2 (Local query)**.** *Let $T_0 \in \mathcal{T}_t$. Breadth-first search over L_t from T_0, in which a frontier pair $\{x, y\}$ is extended by every $w \in N(x) \cup N(y)$ with $\{x, y, w\} \in \mathcal{T}_t$, visits exactly the component of L_t that contains T_0, and evaluates J only on pairs within the closed neighbourhoods of the nodes it reaches.*

Every wedge through a pair $\{x, y\}$ contains a neighbour of x or y, so the search misses no adjacent triple. A node lies in as many communities as there are components containing a triple through it; its full membership follows by seeding from each triple through it not yet visited.

B. Monotone nesting in the threshold

Theorem 1 (Nesting)**.** *If $t_1 \le t_2$, then L_{t_2} is the subgraph of L_{t_1} induced on $\mathcal{T}_{t_2}$, and every community of $\mathcal{C}_{t_2}$ is contained in a community of $\mathcal{C}_{t_1}$.*

Raising t only removes triples, so it only splits communities or removes nodes; it never merges two of them. The components of $\{L_t\}_{t\in[0,1]}$ form a filtration – a forest

of triple components indexed by t – which changes only at the finitely many values J takes; because communities overlap, their node sets need not form a laminar dendrogram. Coarse covers at small t and fragmentation at large t are the two ends of this single hierarchy.

C. The two ends of the hierarchy

Accuracy is scored with the Omega index [78]. For a cover $\mathcal{C}$ let $c_{\mathcal{C}}(i,j)$ be the number of its communities containing both i and j, and $\pi_{\mathcal{C}}(c)$ the fraction of the $\binom{n}{2}$ pairs with $c_{\mathcal{C}} = c$. With ground truth $\mathcal{G}$, $\Omega = (\Omega_u - \Omega_e)/(1 - \Omega_e)$, where $\Omega_u = \Pr[c_{\mathcal{C}} = c_{\mathcal{G}}]$ and $\Omega_e = \sum_c \pi_{\mathcal{C}}(c)\,\pi_{\mathcal{G}}(c)$.

Lemma 1. *If $c_{\mathcal{C}}(i,j) = c$ for every pair and $c_{\mathcal{G}}$ is not identically c, then $\Omega = 0$.*

A cover that gives every pair the same count agrees with the ground truth exactly as often as chance predicts.

Proposition 3 (Endpoints)**.** *Let the ground truth give some pair count 0 and some pair count ≥ 1. (i) If G is connected and $|V| \geq 3$, then $\mathcal{C}_0 = \{V\}$ and $\Omega(0) = 0$. (ii) The communities at $t = 1$ are the classes of nodes with identical closed neighbourhoods having at least three members; if there are none, $\mathcal{C}_1 = \varnothing$ and $\Omega(1) = 0$. Consequently, when the hypotheses of (i) hold and there are no such classes, whenever $\Omega(t) > 0$ for some t the maximum $\Omega^\star = \max_t \Omega(t)$ is attained at an interior threshold $t^\star \in (0,1)$.*

At $t = 0$ every wedge is admitted and the line graph of G connects them; at $t = 1$ only nodes with identical closed neighbourhoods stay together. The proposition does not assert that $\Omega^\star > 0$ or that $\Omega(t)$ has a single maximum; both are empirical. The two hypotheses are separate: (i) needs a connected graph, and the final claim also needs no class of three or more nodes with identical closed neighbourhoods. On noise-free ring lattices the second holds (distinct nodes have distinct arcs when $s \geq 2k+2$), and the first holds when the merges connect all rings, which random merges usually but not always do (a ring that receives no merge is a separate component) and the isolated-merge family with $k = 8$ never does (20 merges cannot connect 40 rings); after rewiring both hold generically. The first fails on the noise-free benchmark without overlap, whose rings are disconnected and recovered exactly at $t = 0$, a case covered by Theorem 2.

D. Exact recovery on overlapping ring lattices

Consider r rings of sizes $s_i \geq 2k+2$, each node joined to its $k/2$ nearest neighbours on either side ($k \geq 4$ even), and a set of *merges*: a node v of one ring is absorbed into a node u of another, which inherits v's edges and so belongs to both ground-truth communities. The merges are *isolated* if they identify disjoint pairs of original nodes from different rings – each original node takes part in at most one merge – and, on each original ring, any two merge positions are more than k apart.

Lemma 2 (Commonality on the lattice)**.** *With isolated merges, let x, y be unmerged nodes at ring distance $d \leq k$, u a merged node at distance d from x along one of its rings, and z, y' neighbours of u on its two different rings. Then*

$$\begin{aligned} J(x,y) = J(d) &\equiv \frac{k+1-d}{k+1+d}, \\ J(u,x) = J_u(d) &\equiv \frac{k+1-d}{2k+1+d}, \\ J(z,y') &= \frac{1}{2k+1}, \end{aligned} \tag{3}$$

and every other pair on distinct rings has $J = 0$. (With ring degrees $k_a \neq k_b$ and x, z on the ring of degree k_a, $J_u(d) = (k_a + 1 - d)/(k_a + k_b + 1 + d)$ and $J(z,y') = 1/(k_a + k_b + 1)$.)

Closed neighbourhoods are arcs of $k+1$ nodes, and a merged node's is the union of two.

Theorem 2 (Exact recovery window)**.** *With at least one merge, all of them isolated, $\mathcal{C}_t$ equals the ground truth if and only if*

$$\begin{aligned} &\frac{1}{2k+1} < t \leq t^+(k), \\ &t^+(k) \equiv \min\Big\{ J_u(1),\, \max\big(J(3), J_u(2)\big) \Big\} \\ &\qquad = \min\Big\{ \frac{k}{2k+2},\, \max\Big(\frac{k-2}{k+4}, \frac{k-1}{2k+3}\Big) \Big\}. \end{aligned} \tag{4}$$

Without merges the condition is $0 \leq t \leq J(2) = (k-1)/(k+3)$.

Explicitly, $t^+(4) = 3/11$, $t^+(6) = 2/5$ and $t^+(k) = k/(2k+2)$ for $k \geq 8$, so the window widens towards $(0, \frac{1}{2})$ as k grows. For a single merge between two rings, exact computation for $k = 4$ to 12 shows that the window also holds for ring sizes down to $s = k+4$ for $k = 4$ and 6, and $s = k+2$ for $k = 8$ to 12; smaller rings widen it upward (for $k = 4$ and $s = 6$ or 7 it is $(1/9, 2/5]$). This covers the rings of Fig. 2 ($s = 9$, $k = 4$).

The two edges of the window have a direct reading. The lower edge is the price of overlap: in the noise-free lattice a node shared by two communities is the only bridge between them, and the bridge is cut once t exceeds the commonality $1/(2k+1)$ of the two communities' members across it. The upper edge is set by the shared nodes themselves. For $k \geq 8$ it is their commonality $J_u(1)$ with either community, roughly halved by their double membership, and above it they drop out of the communities they belong to; for $k = 4$ and 6 it is $\max(J(3), J_u(2)) < J_u(1)$, and above it the triple formed by a shared node and its two nearest neighbours on a

ring splits off as a spurious community. For one isolated merge between rings of even degrees $k_a, k_b \geq 4$ and sizes at least $2k_a+2$ and $2k_b+2$, the same argument gives the window

$$\frac{1}{k_a+k_b+1} < t \leq \min\{U(k_a,k_b), U(k_b,k_a)\},$$
$$U(x,y) = \min\Big\{\frac{x}{x+y+2}, \max\Big(\frac{x-2}{x+4}, \frac{x-1}{x+y+3}\Big)\Big\}, \tag{5}$$

which reduces to Eq. (4) for $k_a = k_b$. Because each merge type imposes its own window, a family whose isolated merges join rings of several degree pairs is recovered exactly if and only if t lies in the intersection of the windows of Eq. (5) over its merge types (for a family in which all these merge types occur). For the two-worlds, three-big and equal-s families this is $(1/17, 4/21]$, against $(1/17, 4/9]$ for uniform rings with $k = 8$; for the gradient it is $(1/17, 4/13]$. Once $k_a \geq 2k_b^2 - 2$ the window of a merge between a dense and a sparse ring no longer overlaps that of merges between two sparse rings, so no single threshold recovers a family that contains both, even without noise. A node formed by merging one node from each of m rings (all other merges isolated) gives the window $(1/(2k+1), \min\{k/(mk+2), \max((k-2)/(k+4), (k-1)/(mk+3))\}]$, whose upper edge tends to $1/m$ as k grows; it is non-empty if and only if $m \leq 2k$. We verified Eqs. (4) and (5) exactly, in rational arithmetic, at and on either side of every distinct commonality value, for $k = 4, \ldots, 24$ and for eleven degree pairs from $(4,4)$ to $(32,8)$, without a mismatch, and the other window statements of this section and of Appendix C by the same exact computation (analysis/exact_windows.py).

Proposition 4 (Shared pairs). *Let A and B be ground-truth communities, and let the pair $\{u_1, u_2\} \subseteq A \cap B$ lie in an admissible triple $T_A \subseteq A$ and in an admissible triple $T_B \subseteq B$. Then T_A and T_B lie in the same component of L_t, whatever the commonality between $A \setminus B$ and $B \setminus A$. Consequently $\mathcal{C}_t$ differs from the ground truth unless some ground-truth community contains $T_A \cup T_B$.*

The two triples share the pair and so are adjacent in L_t. More generally, any admissible triple that shares a pair with an admissible triple inside A and a pair with one inside B joins them. In particular, three communities that pairwise share single nodes u_{AB}, u_{AC}, u_{BC} are joined whenever $\{u_{AB}, u_{AC}, u_{BC}\}$ is admissible and each of its pairs lies in an admissible triple inside the community that contains it, although no two of them share a pair. Two communities that overlap in a pair of nodes close to each other in both are thus read as one, much as k-clique percolation reads communities that share $k-1$ nodes as one [36]; Theorem 2's isolation hypothesis excludes such configurations (Appendix C gives when they fuse two rings). The random merges of our benchmarks are not isolated: with $o_n = 0.05$ ($\lfloor o_n n_0 \rfloor$ merged pairs; Sec. VI A), between 6% and 61% of the rings of a family are fused with another ring just above the lower edge of the window (Table X). With merges placed to satisfy the hypothesis (Sec. VII A), HyperCommon recovers the planted rings exactly at every threshold of our grid inside the window.

Corollary 1 (Pair-disjointness). *At every threshold t, all admissible triples that contain a given pair of nodes lie in one component of L_t. In particular, an edge that lies in an admissible triple is assigned to exactly one community, the community of its triples, and the communities of a node are those of the admitted edges incident to it. The node sets of two communities may still share a pair, even an edge; such a pair lies in admissible triples of at most one of them.*

The corollary describes the kind of overlap HyperCommon represents: no pair of nodes binds two communities (a pair binds a community when it lies in one of its admitted triples), so each admitted edge is assigned to one community and a node's memberships can be read off its admitted edges, as in link communities [34, 35]; overlap needs no separate membership rule and cannot blur two communities into each other through a shared edge. This representation shares an assumption of clique percolation and link clustering that ground-truth studies contradict for many social, collaboration and information networks, where the overlaps of communities are more densely connected than their non-overlapping parts [79, 80]; in product co-purchasing networks such as Amazon, by contrast, overlaps are small [81]. The same property sets the limit of the representation (Proposition 4).

E. Cost

Admissibility is decided for each of the $W = \sum_v \binom{d_v}{2}$ wedges (d_v the degree of v), with deduplication of triangles, from three pair commonalities, all of which one pass over the centred wedges counts. With $m = |E|$, the expected time and the memory are

$$O(n+m+W) = O\big(n[1+\langle d^2\rangle]\big), \tag{6}$$

linear in n when the second moment of the degree distribution is bounded, and $O(n^3)$ for a dense graph. Real networks are typically sparse [2], and large scale-free graphs with unconstrained degrees must be sparse [82]; how often degree tails are heavy enough to make $\langle d^2\rangle$ grow quickly with n is debated [83, 84]. On sixteen of the seventeen real networks of Sec. VII, $W/(m \ln m)$ lies between 0.8 and 13. The exception is YouTube, whose hubs reach degree 28 754 and give $W/(m \ln m) = 33$; the degree-ratio pruning below brings the wedges that are enumerated and scored to $3.7\, m \ln m$. Once the triple scores are stored, every additional threshold costs $O(W)$. Since $J(u,v) \leq \rho(u,v) \equiv (\min(d_u, d_v)+1)/(\max(d_u, d_v)+1)$, a triple that contains an edge with $\rho < t_{\min}$ has score below $t_{\min}$ and can be discarded before scoring without changing $\mathcal{T}_t$, and hence $\mathcal{C}_t$, for any

$t \geq t_{\min}$, provided the commonalities of the pairs that remain are computed from the complete neighbourhoods (Appendix C). Sec. VII reports measured running times. A local query examines the frontier candidates of the community it grows, including rejected ones, and reads their neighbourhoods, so its cost scales with that neighbourhood rather than with the whole graph.

VI. BENCHMARKS AND EVALUATION PROTOCOL

A. Planted overlapping ring lattices

Following Refs. [25, 85], each planted community is a ring lattice: s nodes on a circle, each joined to its $k/2$ nearest neighbours on either side; communities may differ in size s and in degree k. Overlap is planted by merging $\lfloor o_n n_0 \rfloor$ pairs of nodes from different rings, each node in at most one merge (n_0 is the number of nodes before the merges): the survivor belongs to both rings in the ground truth and keeps both neighbourhoods, with twice the ring degree – the case most favourable to commonality. Noise rewires a fraction $p \in \{0, 0.05, \ldots, 0.8\}$ of the original edges: each keeps one random endpoint and moves the other to a non-neighbour chosen uniformly or preferentially (Appendix B); at $p = 0$ the communities are exact. Every (p, seed) pair yields an independently generated graph.

The ring families (Table VIII) span equal and unequal s and k, from uniform rings ($n_0 = 2000$, $(s, k) \in \{(50, 8), (100, 16)\}$, $o_n \in \{0, 0.05, 0.10\}$) to "two worlds" (five dense rings of 200 nodes, twenty sparse rings of 50) and a gradient of ten rings from 600 to 30 nodes. All six unequal families are also generated at $o_n = 0$ and 0.10, so that, with the uniform and preferential families, the benchmark has 28 ring settings. Numbers quoted below for a ring family without its overlap level are at $o_n = 0.05$. Further families isolate one question each (Appendix B): *geometry* families rewire a fraction q of each ring's edges inside the ring, moving each community from a lattice ($q = 0$) through clustered small-world graphs to a random graph ($q = 1$) at fixed sizes, mean degrees and overlap; *isolated-merge* families ($o_n = 0.01$) place the merges more than $2k$ apart on every ring, within the hypothesis of Theorem 2; and control sets change one assumption at a time, with rings that contain *hubs*, *split* overlap (a shared node keeps about $k/2$ edges in each ring, as in LFR graphs), *random geometric* communities (clustered but not lattices), and two *null* graphs with triadic closure but no communities, a random geometric graph and a Holme–Kim graph [86].

B. Overlapping LFR graphs

We use the reference implementation of the overlapping Lancichinetti–Fortunato–Radicchi benchmark [47, 48] with $N = 2000$, a fraction $o_n \in \{0, 0.1, 0.3\}$ of nodes in $o_m = 2$ communities, and mixing parameter $\mu \in \{0.1, \ldots, 0.6\}$. Three slices at $o_n = 0.1$ change one factor at a time (degree, o_m, community sizes), and two further families at $o_n = 0.1$ raise the clustering with the generator's own rewiring ($C_{\text{LFR}} = 0.3,\ 0.5$); all settings are in Appendix B.

C. Methods compared

HyperCommon is compared with its ablations, with the methods closest to it, and with methods from the main families of overlapping and non-overlapping detection; BigClam, ego-splitting and the local-fitness methods LFM and OSLOM are not included (LFM because the CDLIB implementation took more than ten minutes on a single 2000-node benchmark graph, against the thousands of graphs of the benchmark):

- **Ablations**, each removing one ingredient: *triangles only* admits only closed triples (J-thresholded 3-clique percolation, verified numerically); *two-test* admits an open wedge when its two edges pass, without testing its non-adjacent pair; *edge-connected* joins HyperCommon's triples only through shared edges rather than through any shared pair.
- **Link communities** [34], with the same similarity, cut at every value of HyperCommon's threshold grid.
- **Clique percolation** (CPM) [36].
- **SCAN** [33] with cosine similarity (SCAN) and with Jaccard similarity (SCAN-J).
- **Leiden** [9] optimising the Reichardt–Bornholdt (RB) modularity [87] and, separately, the Constant Potts Model [61] (L-CPM); **Infomap** [10], two-level, over a range of Markov times [67, 68]; **Walktrap** [66]; **label propagation** (LPA) [16].
- **Stochastic block model** (SBM): the nested degree-corrected stochastic block model [11], fitted by minimum description length with GRAPH-TOOL [88], which selects its own number of groups; each level of the fitted hierarchy is a partition, treated as a parameter value, the finest being the model's own choice. It was run on the benchmark graphs, not on the real networks.
- **Overlapping label and ego methods**: SLPA [18], DEMON [64] and ANGEL [65].

Each parameterised method has a grid (Table IX). In a preliminary run, the best parameter of clique percolation, SLPA, DEMON, ANGEL and SCAN lay at an end of its grid on 20–65% of the graphs; their grids were then widened and fixed before any of the results reported

here were computed. Table S18 reports how often the selected parameter still lies at an end of its grid: on at most 5% of the test graphs for HyperCommon and its edge-connected and two-test variants, link communities, the two Leiden objectives, Infomap and SCAN under every protocol; on 10%–27% for the triangles-only variant, DEMON, ANGEL and SCAN-J under the oracle and label-free protocols (at most 17% frozen); on 38%–49% for clique percolation, almost always at clique size 3, the smallest possible; and on 20%–75% for SLPA, mostly at $r = 0.01$ under the oracle and frozen protocols and at $r = 0.45$ under the label-free one, whose grid was not extended further. Implementations and versions are listed in Appendix B.

D. Four protocols

Every method, HyperCommon included, is scored under the same four protocols. *Oracle*: on each test graph, each method is credited with its best parameter on that graph, which measures what its family of covers can achieve, with the same access to the ground truth for all. *Label-free*: on each test graph, each method takes the parameter whose cover has the largest overlapping modularity [62],

$$Q_{\mathrm{ov}} = \frac{1}{2m} \sum_{c} \sum_{u,v \in c} \frac{1}{O_u O_v} \Big(A_{uv} - \frac{d_u d_v}{2m} \Big), \qquad (7)$$

where O_u is the number of communities that contain u, nodes in no community contribute nothing (our extension to partial covers), and m and d_u are taken over the whole graph. The criterion uses the graph alone and measures what the method delivers without labels (ties are broken by grid order). For a partition $O_u = 1$ for every node, and Eq. (7) is the Newman–Girvan modularity; for Leiden with either objective and for Infomap this protocol therefore selects the partition of largest modularity along each parameter path (Table S17). *Frozen*: for each family and method, the parameter with the largest mean Omega over three calibration graphs per noise level, all noise levels pooled, is fixed and applied to ten fresh test graphs per noise level. *Native*: each method at its own default or selection rule where it has one: Leiden RB at its default $\gamma = 1$, the Newman–Girvan modularity itself; L-CPM at γ equal to the edge density $2m/[n(n-1)]$ [61]; link communities at the cut of largest partition density [34]; Infomap at its default Markov time 1; SLPA at $r = 0.1$, the default of CDLIB; the stochastic block model at the finest level of its hierarchy; and the label-free choice for HyperCommon and the methods without a default. For the Constant Potts Model, link communities, Infomap, SLPA and the stochastic block model the native choice does not involve modularity; Leiden, Walktrap, HyperCommon and the methods without a default keep a modularity-based choice.

E. Metrics and statistics

Covers are compared with the adjusted Omega index [78]; the overlapping normalised mutual information of McDaid *et al.* with max normalisation (ONMI) [89]; the best-match F1 score [72], averaged over both matching directions; a per-community recovery score, the mean over true communities of the Jaccard similarity with the best-matching detected community, also reported for the communities no larger and larger than the median size; and coverage, the fraction of nodes in at least one community. Omega counts node pairs and is dominated by the largest communities; the per-community score weighs every community equally. An unassigned node has pair count zero with every other node in Omega and is a singleton in ONMI, so both penalise leaving nodes out. Each metric is averaged over the noise range, $\bar{\Omega} = \int \Omega(p)\, dp / \int dp$, per test seed; uncertainties are 95% bootstrap intervals over the ten test seeds, and differences between methods are paired by seed.

VII. RESULTS

A. Exact recovery and the threshold

An exact implementation confirms Theorem 2. On rings whose merges satisfy its isolation hypothesis ($n_0 = 2000$, $o_n = 0.01$, merges more than $2k$ apart), without noise HyperCommon returns the planted rings exactly, $\Omega = 1$ on every test graph, at each of the 20 thresholds of our grid inside the window for $k = 8$ and the 22 inside it for $k = 16$, and at no threshold outside it ($\Omega < 0.98$). With a fifth of the edges rewired, the mean Ω at the best threshold of each graph is still 1.000 for $k = 16$ and 0.991 for $k = 8$. Without the third-pair test the same graphs are never recovered exactly ($\Omega < 0.98$ without noise, a mean of 0.74 at $p = 0.2$ for $k = 8$): the two edges of the wedge that crosses a merge have commonality $J_u(1)$, which is at least every threshold of the window, so the pair rule admits that wedge throughout it.

When merges fall at random, the isolation hypothesis fails: at $o_n = 0.05$ each ring carries on average 5 ($k = 8$) or 10 ($k = 16$) merge positions, whereas at most five positions more than k apart fit on a ring of 50 nodes with $k = 8$ or of 100 nodes with $k = 16$. Rings are then fused inside the window through nearby merged nodes (Proposition 4; Table X), which keeps the noise-free curve of Fig. 3(a) ($k = 8$) between 0.92 and 0.99 across the window, while for $k = 16$ [Fig. 3(b)] it reaches 0.95 only for $t \geq 0.30$. Otherwise each curve rises from near zero at small t, where communities coalesce, to a maximum and decays as they fragment, its covers nested (Theorem 1). As noise grows the best threshold moves down towards $t \approx 0.1$, so a threshold has to be chosen per graph, and the overlapping modularity of the cover chooses it well for HyperCommon: the label-free threshold attains at least 95% of the best threshold's mean Ω on 27 of the 36

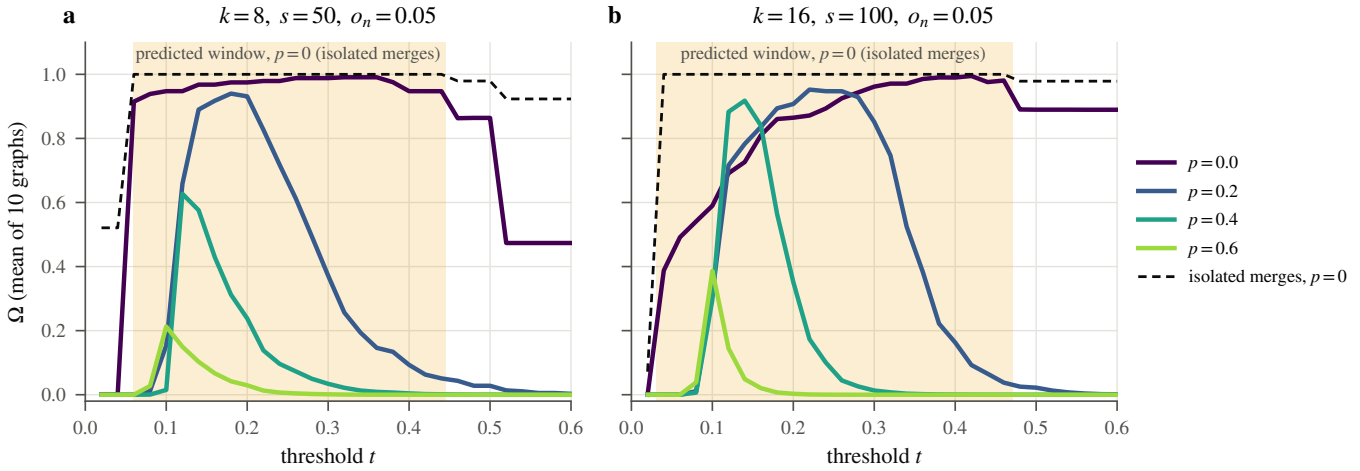


FIG. 3. Accuracy of HyperCommon as a function of the threshold on overlapping uniform rings ($n_0 = 2000$, $o_n = 0.05$), one curve per rewiring level p, each the mean over ten test graphs. Shaded: the exact-recovery window that Theorem 2 predicts for $p = 0$ when merges are isolated; the plotted graphs' random merges do not satisfy the isolation hypothesis. Black dashed: graphs with isolated merges ($o_n = 0.01$, merges more than $2k$ apart) at $p = 0$, which reach $\Omega = 1$ at every grid threshold inside the window and at none outside it.

benchmark settings and at least 90% on 31 (Supplemental Material [75], Tables S1 and S3).

B. Planted rings

Label-free protocol. Table I summarises the four protocols of Sec. VI D. Under the label-free one, every parameter is chosen from the graph alone by the overlapping modularity of Eq. (7), which for a partition is the Newman–Girvan modularity (Table S1). Over the 28 ring settings HyperCommon has the highest Ω in 16, by paired 95% intervals that exclude zero (ablations not counted), is not significantly different from the best other method in 5 and trails in 7: on dense rings with little overlap, where Leiden leads (0.84 against 0.78 at $k = 16$, $o_n = 0$); on rings of equal size and unequal degree at $o_n = 0.10$, where the Constant Potts Model leads (0.63 against 0.61); and on the gradient at all three overlap levels, where link communities lead. The margins are largest where community sizes differ most and the communities overlap: $\Omega = 0.78$ against 0.68 for clique percolation on three big, many small, and 0.76 against 0.66 on two worlds [Fig. 4(c,d)].

Selector and native protocol. Part of these margins lies in the selection of the parameter rather than in the objectives: at its best resolution the Constant Potts Model reaches 0.71 on three big, many small and 0.78 on two worlds (Table S3), against 0.34 and 0.64 when its resolution is chosen by modularity, while HyperCommon goes from 0.80 to 0.78 and from 0.78 to 0.76 (Table S17). The native protocol runs each method at its own default or selection rule, while HyperCommon keeps its label-free threshold (Table S5). Under it HyperCommon is ahead in 17 of the 28 ring settings, level in 5 and behind in 6, on the dense rings with little overlap and on the gradient. The Constant Potts Model gains most from its own rule and is level with HyperCommon on two worlds at $o_n = 0.05$ and 0.10; under preferential noise, by contrast, the edge density serves it worse than the resolution of largest modularity (0.52–0.56 against 0.66), and HyperCommon stays well ahead (0.77–0.79).

Oracle protocol. Credited with its best parameter on each graph (Table S3), every method narrows the gap: over the 28 ring settings HyperCommon is ahead in 11, level with the best other method in 4 and behind in 13. It stays ahead on three big, many small at every overlap level (0.80 against 0.72 for link communities at $o_n = 0.05$) and on the gradient with overlap (0.57 against 0.54); where it is behind, Leiden with one objective or the other leads, except on the gradient without overlap, where link communities do. Modularity optimisation with a single resolution fails on the gradient even without noise: at $p = 0$ Leiden attains $\Omega = 0.28$ there, and 0.26 at the resolution at which it returns the right number of communities, because small rings are merged while large ones are split [59, 90]; HyperCommon attains 0.99.

Frozen protocol. With one parameter per family fixed on calibration graphs, all noise levels pooled (Table S4), the two Leiden objectives, whose best resolutions vary little with noise, are the most accurate on most ring families, and HyperCommon is ahead in 1 of the 28 settings and not significantly different in 5: its best threshold moves with noise (Fig. 3), so a single threshold fits the noise-free graphs or the noisy ones but not both. Choosing the threshold per graph, as the label-free protocol does, removes this limitation without labels on every ring family except the gradient.

The gradient. Here, at $o_n = 0.05$, the selector picks a threshold above the oracle's on 113 of the 170 test graphs

TABLE I. Scorecard of the benchmarks under the four protocols: Ω of HyperCommon minus that of the best other method (ablations excluded), mean over the noise range ($p \in [0, 0.8]$ for rings, $\mu \in [0.1, 0.6]$ for LFR), ten test graphs per noise level, with the best other method in brackets when HyperCommon is not significantly ahead of it and its difference rounds to 0.00 or below. Mark: the paired 95% bootstrap interval of the difference over the ten test seeds lies above zero (+), contains zero (=) or lies below zero (−); intervals and the second-best method in Table S16. Oracle: best parameter per graph; label-free: largest overlapping modularity, Eq. (7); frozen: one parameter per family from calibration graphs; native: each method's own default or selection rule where it has one (Sec. VI), the label-free choice otherwise. Counts are not corrected for multiplicity. Method abbreviations as in Table S1; the full tables are Tables S1 and S3–S5.

Family	o_n	oracle	label-free	frozen	native
uniform, k=8	0.00	-0.05^{-} (Leiden)	$-0.00^{=}$ (Leiden)	-0.08^{-} (Leiden)	$-0.00^{=}$ (Leiden)
	0.05	$+0.00^{+}$	$+0.02^{+}$	-0.03^{-} (Leiden)	$+0.06^{+}$
	0.10	$+0.01^{+}$	$+0.05^{+}$	$+0.01^{=}$	$+0.06^{+}$
uniform, k=16	0.00	-0.08^{-} (Leiden)	-0.06^{-} (Leiden)	-0.10^{-} (Leiden)	-0.06^{-} (Leiden)
	0.05	-0.04^{-} (Leiden)	-0.03^{-} (Leiden)	-0.19^{-} (Leiden)	-0.04^{-} (Leiden)
	0.10	$-0.01^{=}$ (Leiden)	$+0.00^{=}$ (Leiden)	-0.32^{-} (Leiden)	$+0.00^{=}$ (Leiden)
equal k, unequal s, k=8	0.00	-0.03^{-} (Leiden)	$+0.01^{+}$	-0.04^{-} (Leiden)	$+0.01^{+}$
	0.05	$+0.01^{+}$	$+0.03^{+}$	$+0.00^{=}$ (Link)	$+0.09^{+}$
	0.10	$+0.00^{=}$ (Link)	$+0.04^{+}$	$+0.00^{=}$ (CPM)	$+0.08^{+}$
equal k, unequal s, k=16	0.00	-0.07^{-} (Leiden)	-0.04^{-} (Leiden)	-0.10^{-} (Leiden)	-0.04^{-} (Leiden)
	0.05	-0.04^{-} (Leiden)	$+0.00^{=}$ (Leiden)	-0.17^{-} (Leiden)	$+0.01^{+}$
	0.10	$+0.01^{+}$	$+0.04^{+}$	-0.31^{-} (Leiden)	$+0.04^{+}$
equal s, unequal k	0.00	-0.03^{-} (Leiden)	$-0.01^{=}$ (L-CPM)	-0.07^{-} (Leiden)	$+0.00^{=}$ (Leiden)
	0.05	-0.01^{-} (L-CPM)	$-0.01^{=}$ (L-CPM)	-0.18^{-} (Leiden)	$+0.02^{+}$
	0.10	-0.03^{-} (L-CPM)	-0.02^{-} (L-CPM)	-0.33^{-} (L-CPM)	$+0.03^{+}$
two worlds	0.00	-0.00^{-} (L-CPM)	$+0.03^{+}$	-0.05^{-} (L-CPM)	$+0.01^{+}$
	0.05	-0.01^{-} (L-CPM)	$+0.10^{+}$	-0.25^{-} (L-CPM)	$+0.00^{=}$ (L-CPM)
	0.10	$-0.00^{=}$ (L-CPM)	$+0.06^{+}$	-0.38^{-} (L-CPM)	$+0.01^{=}$
three big, many small	0.00	$+0.01^{+}$	$+0.03^{+}$	$+0.03^{+}$	$+0.09^{+}$
	0.05	$+0.07^{+}$	$+0.10^{+}$	-0.16^{-} (CPM)	$+0.10^{+}$
	0.10	$+0.04^{+}$	$+0.06^{+}$	-0.23^{-} (CPM)	$+0.06^{+}$
gradient	0.00	-0.01^{-} (Link)	-0.19^{-} (Link)	$-0.00^{=}$ (Link)	-0.14^{-} (CPM)
	0.05	$+0.03^{+}$	-0.16^{-} (Link)	$-0.02^{=}$ (ANGEL)	-0.11^{-} (ANGEL)
	0.10	$+0.03^{+}$	-0.15^{-} (Link)	-0.14^{-} (DEMON)	-0.12^{-} (CPM)
uniform k=8, α=1	0.05	$+0.01^{+}$	$+0.04^{+}$	-0.03^{-} (Leiden)	$+0.08^{+}$
uniform k=8, α=2	0.05	$+0.01^{+}$	$+0.02^{+}$	-0.02^{-} (Leiden)	$+0.08^{+}$
two worlds, α=1	0.05	-0.05^{-} (L-CPM)	$+0.11^{+}$	-0.29^{-} (L-CPM)	$+0.12^{+}$
two worlds, α=2	0.05	$+0.00^{=}$ (L-CPM)	$+0.13^{+}$	-0.20^{-} (L-CPM)	$+0.15^{+}$
LFR, o_n=0		-0.14^{-} (Leiden)	-0.16^{-} (Info.)	-0.18^{-} (Leiden)	-0.16^{-} (Info.)
LFR, o_n=0.1		-0.15^{-} (L-CPM)	-0.18^{-} (Info.)	-0.24^{-} (Info.)	-0.19^{-} (Info.)
LFR, o_n=0.3		-0.21^{-} (SLPA)	-0.19^{-} (Info.)	-0.40^{-} (Leiden)	-0.21^{-} (SLPA)
LFR, $\langle k \rangle$=10		-0.33^{-} (Leiden)	-0.28^{-} (Info.)	-0.37^{-} (Leiden)	-0.28^{-} (Info.)
LFR, o_m=4		-0.15^{-} (SLPA)	-0.09^{-} (Info.)	-0.18^{-} (SLPA)	-0.10^{-} (SLPA)
LFR, sizes 50–250		-0.45^{-} (Leiden)	-0.45^{-} (Leiden)	-0.55^{-} (Leiden)	-0.43^{-} (Leiden)
LFR, C_{LFR}=0.3		-0.23^{-} (Leiden)	-0.23^{-} (SLPA)	-0.31^{-} (Leiden)	-0.25^{-} (SLPA)
LFR, C_{LFR}=0.5		-0.08^{-} (Leiden)	-0.11^{-} (LPA)	-0.14^{-} (SLPA)	-0.12^{-} (SLPA)
+/=/−, rings		11/4/13 of 28	16/5/7 of 28	1/5/22 of 28	17/5/6 of 28
+/=/−, LFR		0/0/8 of 8	0/0/8 of 8	0/0/8 of 8	0/0/8 of 8

and below it on one, so the selected cover is too fine (181 communities on average over the noise range against 147 at the best threshold and 10 planted; Tables S13 and S15); at every overlap level it attains only 49%–64% of the best threshold's Ω.

Per-community recovery and ONMI. Because Ω counts node pairs, it is dominated by the largest communities. On the two families with a few large dense rings and many small sparse ones, at $o_n = 0.05$, HyperCommon recovers the large rings best (0.84–0.85 against 0.72–0.83 for the Constant Potts Model; Tables S10 and S11) but splits the small sparse rings, which the Constant Potts Model recovers best (0.53–0.54 against 0.33–0.38); with labels the Constant Potts Model therefore leads there by ONMI and by recovery averaged over all communities (Tables S6 and S7). Over the 28 ring settings HyperCommon leads by ONMI on 10 with labels and on 15 without (Table S8).

Overlap level. With labels, HyperCommon's margin over the Constant Potts Model grows with overlap on three big, many small (0.03, 0.09 and 0.23 at $o_n = 0$, 0.05 and 0.10), while on two worlds the Constant Potts Model is ahead by less than 0.01 at every overlap level

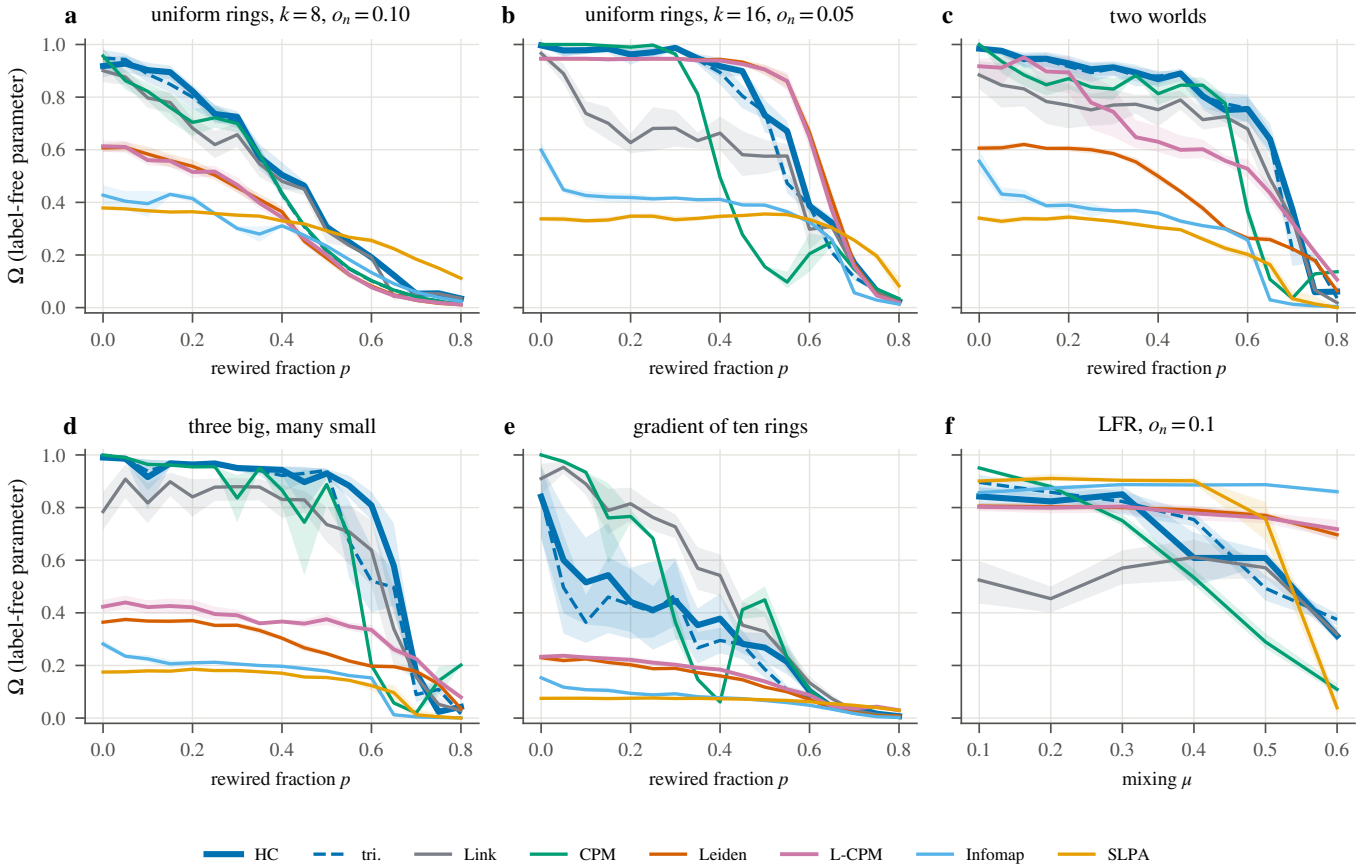


FIG. 4. Accuracy against noise with every method's parameter chosen without labels (largest overlapping modularity on each graph); lines are means over ten test graphs and bands 95% bootstrap intervals. (a,b) Uniform rings; (c–e) rings of unequal size and density ($o_n = 0.05$); (f) overlapping LFR graphs, against the mixing parameter. In (e) the selector picks too fine a cover for HyperCommon (Sec. VII B). The same panels under the oracle protocol are in Fig. S1.

(significantly at $o_n \leq 0.05$).

Robustness to noise. Without labels, HyperCommon's $p_{1/2}$, the rewired fraction at which its mean Ω first falls below one half (Table S2), is the latest of all methods, or within 0.01 of it, in ten of the sixteen ring families; on the four families whose large rings have degree 32 and small rings degree 8 it has the highest mean Ω over the high-noise half, $p \in [0.4, 0.8]$ (0.60 against 0.49 for link communities on three big, many small), and on the gradient it is the least robust of the leading methods. Degree-biased noise affects it little, as it does most other methods.

C. Ablations and the closest relatives

Each ingredient of the construction was removed in turn (Sec. VI C); a difference is counted when its paired 95% interval over the test graphs excludes zero. Testing the third, non-adjacent pair of a wedge matters most: without it Ω falls in all 36 benchmark settings under the oracle and the label-free protocol, by 0.06–0.34 on rings and up to 0.42 on LFR graphs, because a wedge whose two edges pass then reaches into another community whenever its centre has a neighbour there. Admitting open wedges improves on admitting only triangles (J-thresholded 3-clique percolation) in 26 and 25 of the 36 settings under the oracle and the label-free protocol, most where triangles are scarce (0.04–0.11 on sparse rings and 0.08–0.10 on the sparse LFR slice); triangles alone are better in one setting under the oracle protocol and in none under the label-free one. With a single threshold frozen per family (Table S4) the order reverses on the unequal families: triangles alone have the higher mean Ω in 15 of the 36 settings, including two worlds under the three noise models and three big, many small. Joining triples only through shared edges makes little difference on rings when the parameter is chosen per graph (within 0.02 in mean Ω) and is mostly slightly better on the LFR graphs (by up to 0.03 with labels and 0.05 without).

Link communities with the same similarity cut at every threshold [34], whose edge similarity is the commonality of the wedge's third pair, are less accurate than HyperCommon in 25 of the 28 ring settings under the oracle and the label-free protocol. With labels they are level with it on uniform rings with $k = 16$ without overlap

and on rings of equal degree and unequal size with $k = 8$ at $o_n = 0.10$, and more accurate on the gradient without overlap; without labels they are more accurate on the gradient at every overlap level. Requiring the wedge's two edges to pass as well, as the triple rule does, is what separates the two methods most where communities overlap (0.10–0.23 in label-free Ω on overlapping rings of degree 16 or of unequal density, the gradient excepted). On Amazon, by contrast, the two-test variant is the more accurate by F1 (Sec. VII F).

Commonality function. Among the members of the commonality family, with the best threshold per graph, Jaccard is the best, or within 0.01 of it at the precision of Table S21, on seventeen of eighteen benchmark settings; the label-free comparison is in Table S22.

D. LFR graphs

Overlapping LFR graphs are the complementary regime [Fig. 4(f)]: Leiden, Infomap, SLPA and, without labels, the stochastic block model are the most accurate there, and HyperCommon trails on all eight LFR families under every protocol, matching Leiden and Infomap only at low mixing and with its best threshold. The pairwise signal is not what fails: the area under the ROC curve of the commonality of edges inside against across planted communities is 0.97 at $\mu = 0.3$, and wedges that span two communities pass the three-pair test at $t = 0.1$ with probability below 10^{-4}. What fails is connectivity inside large communities. LFR communities are random graphs internally, with degree independent of size, so a large community is sparse and its members share few neighbours: at $\mu = 0.3$, 89% of the wedges inside communities of at most 40 nodes pass at $t = 0.1$, but only 21% of those inside communities of at least 80 nodes. Percolation then breaks large communities into pieces at any threshold that keeps communities apart, which is why HyperCommon's deficit is largest with large communities (0.42 against 0.87 for Leiden). Raising the clustering with the generator's own rewiring does not help, since it adds triangles across communities as well.

E. Lattice and random communities

The LFR results suggest that the internal geometry of communities, more than their size, limits where HyperCommon is the right description. The geometry families test this (Table II): sizes, mean degrees and overlap are held fixed while a fraction q of each ring's edges is rewired inside the ring, so that each community moves from a lattice, in which two adjacent members share up to $k - 2$ neighbours, to a random graph, in which they share about $(k - 1)^2/s$.

On lattice communities HyperCommon is ahead of every other method with labels in three of the four families and level with the best in the fourth, and without labels it is ahead on three. The advantage is specific to this geometry. With labels it is gone once a tenth of each community's edges are rewired internally, while the communities are still strongly clustered (about 0.5); without labels HyperCommon is still ahead on three big, many small at $q = 0.1$. From $q = 0.3$ on, the best other method is ahead in every family under both protocols, with labels by 0.25 on the uniform rings at $q = 1$ (0.60 against 0.85). As on LFR graphs, members of a random community share few neighbours, so few triples pass at a threshold that keeps communities apart, and the communities break into pieces. The change of degrees that moving edge endpoints causes is not the reason: with degree-preserving double-edge swaps instead (Table S19), HyperCommon's Ω is within 0.01 of its value under endpoint moves at $q = 0.1$ and 0.3, on the uniform rings and on three big, many small, under both protocols. Open wedges slow the decline: with labels, on the uniform rings triangles alone fall from 0.65 to 0.29 as q goes from 0 to 1, and HyperCommon from 0.73 to 0.60.

Three further controls change one assumption at a time (Table S19): hubs inside the communities move HyperCommon's Ω by at most 0.03, though Leiden gains and leads on the dense hub rings, and the Constant Potts Model on two worlds with hubs; split overlap, in which a shared node divides its edges between its rings as in LFR graphs, keeps it within 0.05 of the merged-node families, so the way LFR graphs share nodes is not what limits it on two worlds and three big, many small; random geometric communities, clustered but not lattices, leave it behind or level with the best other method. On graphs with triadic closure but no planted communities (Table S20) every method reports communities, label-free HyperCommon 101 on the random geometric graph and 175 on the Holme–Kim graph: neither it nor its label-free threshold tells such a graph from one with planted communities, and a test against randomised graphs that keep the degrees and the clustering would have to be added.

F. Real networks

Table III reports detection on the full Amazon, DBLP and YouTube graphs [72]; SCAN, Walktrap, DEMON and ANGEL were not run on these graphs, and clique percolation and SLPA only on the induced subgraphs (Table S23). Scored against the distinct node sets instead (Table S24), F1 changes by at most 0.03 (at most 0.01 for HyperCommon and its variants) and the order of the methods in each column is unchanged. HyperCommon's threshold is chosen without labels at little cost in F1: the label-free F1 is 99.9%, 94% and 89% of its best F1 on the three graphs (ratios of the unrounded values). The same selector serves the modularity-based methods less well: Leiden falls to 0.53, 0.29 and 0.19 in F1, about half of its best values, and the Constant Potts Model to 0.56, 0.24 and 0.25. Under this selector Infomap is the most accurate on Amazon and YouTube (F1 0.89

TABLE II. Internal geometry: before the noise, a fraction q of each ring's edges is rewired inside the ring, from lattice rings ($q = 0$) to rings that are random graphs ($q = 1$), at fixed size and mean degree; $o_n = 0.05$. Uniform: 40 rings of 50 nodes, $k = 8$; two worlds: five rings of 200 nodes ($k = 32$) and twenty of 50 ($k = 8$); three big, many small: three of 400 ($k = 32$) and sixteen of 50 ($k = 8$); gradient: ten rings of 600 to 30 nodes ($k = 16$ to 8). Mean Ω over $p \in \{0, 0.1, \ldots, 0.6\}$, ten test graphs per noise level, under the oracle and label-free protocols, for HyperCommon (HC), its triangles-only variant and the best other method; C: mean clustering of the noiseless graphs (ten seeds); cov.: coverage of HC's label-free cover. Superscripts on HC: paired 95% interval of its difference from the best other method excludes zero ($^+$ HC ahead, $^-$ behind). Bold: HC best or within 0.005 of the best other method. Means are over $p \in \{0, 0.1, \ldots, 0.6\}$, so the $q = 0$ entries differ from those of Tables S1 and S3 for the same families, which average over $p \in [0, 0.8]$.

			oracle			label-free			
Family	q	C	HC	tri.	best other	HC	tri.	best other	cov.
uniform, k=8	0.0	0.62	**0.73**$^+$	0.65	0.73 (Link)	**0.70**$^+$	0.64	0.67 (Link)	0.98
	0.1	0.48	0.68$^-$	0.56	0.81 (Leiden)	0.65	0.56	0.66 (Leiden)	0.97
	0.3	0.28	0.62$^-$	0.41	0.85 (Leiden)	0.57$^-$	0.41	0.73 (Infomap)	0.95
	0.6	0.16	0.61$^-$	0.28	0.86 (Leiden)	0.55$^-$	0.28	0.77 (Infomap)	0.96
	1.0	0.15	0.60$^-$	0.29	0.85 (Leiden)	0.55$^-$	0.28	0.77 (Infomap)	0.95
two worlds	0.0	0.67	**0.91**	0.90	0.91 (L-CPM)	**0.89**$^+$	0.89	0.82 (CPM)	0.94
	0.1	0.51	0.88$^-$	0.87	0.94 (L-CPM)	0.86$^-$	0.86	0.91 (L-CPM)	0.93
	0.3	0.30	0.84$^-$	0.81	0.95 (L-CPM)	0.82$^-$	0.80	0.92 (L-CPM)	0.94
	0.6	0.17	0.72$^-$	0.68	0.95 (L-CPM)	0.69$^-$	0.66	0.92 (L-CPM)	0.91
	1.0	0.16	0.66$^-$	0.66	0.95 (L-CPM)	0.62$^-$	0.66	0.92 (L-CPM)	0.92
three big, many small	0.0	0.68	**0.94**$^+$	0.93	0.87 (CPM)	**0.93**$^+$	0.91	0.85 (CPM)	0.90
	0.1	0.51	0.92$^-$	0.91	0.95 (L-CPM)	**0.91**$^+$	0.91	0.83 (CPM)	0.91
	0.3	0.28	0.84$^-$	0.80	0.96 (L-CPM)	0.83$^-$	0.79	0.95 (L-CPM)	0.91
	0.6	0.13	0.51$^-$	0.61	0.96 (L-CPM)	0.50$^-$	0.56	0.95 (L-CPM)	0.83
	1.0	0.11	0.28$^-$	0.54	0.96 (L-CPM)	0.27$^-$	0.45	0.95 (L-CPM)	0.80
gradient of ten rings	0.0	0.66	**0.74**$^+$	0.72	0.70 (Link)	0.42$^-$	0.37	0.64 (Link)	0.93
	0.1	0.49	0.65$^-$	0.63	0.73 (L-CPM)	0.43$^-$	0.38	0.55 (Link)	0.93
	0.3	0.25	0.44$^-$	0.43	0.86 (L-CPM)	0.40$^-$	0.34	0.76 (Walktrap)	0.93
	0.6	0.08	0.16$^-$	0.11	0.88 (L-CPM)	0.16$^-$	0.09	0.86 (L-CPM)	0.87
	1.0	0.04	0.11$^-$	0.02	0.87 (L-CPM)	0.09$^-$	0.01	0.85 (L-CPM)	0.89

and 0.34) and label propagation on DBLP (0.51), with label-free HyperCommon within 0.02–0.05 on each. By Ω the modularity- and flow-based partitions lead on all three. Detecting on the subgraph induced by the labelled nodes, as is common [72], makes F1 uninformative on Amazon and YouTube: there the connected components already score within 0.005 of the best method, while Ω still separates them.

Table IV extends the comparison to fourteen smaller networks from the Netzschleuder repository [91]: ten Facebook ego networks whose ground truth is the user's own, overlapping, social circles [92], college football conferences [1], the leaning of political books [93, 94] and of political blogs [95], and the classes of a primary school [96]; recorded groups of fewer than three nodes are discarded. With Jaccard commonality and its best threshold, HyperCommon is more accurate than every other method on college football and tied on political blogs. Its mean rank among the sixteen methods of Tables S25–S29 is 8.3 without labels and 7.3 with labels, mid-field, and the most accurate method varies from network to network, consistent with the absence of a universally best method [21]. With labels choosing among 700 (function, threshold) pairs per network – an upper bound rather than the analogue of one parameter grid – HC_f is the most accurate method on four networks by F1 and on four by Ω (Table V), and on the primary school the choice of function alone raises HyperCommon's F1 from 0.41 to 0.83. Testing the third pair helps less here than on the benchmarks: with the best threshold the two-test variant has the higher F1 on nine of the fourteen networks. Without labels, overlapping modularity is a weaker guide on these small networks than on the large ones: the label-free threshold retains 95% of the best F1 or more on seven of the fourteen, and on political blogs, a two-group network in which HyperCommon's best cover is the giant component, it selects fifteen small communities instead. Over all seventeen real networks the label-free threshold retains a median 94% of the best F1.

G. Running time and scale

HyperCommon scores every wedge once and then reads off the cover at any threshold, so a sweep of its threshold costs one scoring plus one component search per threshold. On a personal computer (Intel Core Ultra 9 185H, 32 GB; Fig. 5) our vectorised implementation (Python

TABLE III. Detection on the full SNAP Amazon (334 863 nodes), DBLP (317 080) and YouTube (1 134 890) graphs, scored on the nodes of their ground-truth communities (top 5000 for Amazon and DBLP, all groups of at least three nodes for YouTube), to which every detected community is restricted. Best-match F1 against the recorded communities and Ω against their distinct node sets, at each method's best parameter by that metric (best) and at the parameter chosen without labels by maximal overlapping modularity on the full graph (free); cov.: fraction of the scored nodes in at least one community at the free parameter. HC_f: the seven commonality functions together, the free choice taken over all of them. Leiden at $\gamma = 1$ and Infomap at Markov time 1 are the two methods at their defaults (one parameter, so best and free coincide). Connected components are a trivial reference. On YouTube HyperCommon uses degree-ratio pruning, exact for every $t \geq 0.02$ (Sec. V E), which excludes link communities and the other functions; clique percolation and SLPA are compared on the induced subgraphs (Table S23), and SCAN, Walktrap, DEMON and ANGEL were not run on these graphs, to bound the computing time of the large runs. Stochastic methods: one seed. F1 against the distinct node sets is in Table S24; it changes F1 by at most 0.03 (at most 0.01 for HyperCommon and its variants) and reverses the order of no two methods in a column. The top-5000 lists of Amazon and DBLP frequently consist of near-cliques [54], which can favour methods that find small dense groups. Bold: best in column.

	Amazon					DBLP					YouTube				
	F1 best	F1 free	Ω best	Ω free	cov.	F1 best	F1 free	Ω best	Ω free	cov.	F1 best	F1 free	Ω best	Ω free	cov.
components	0.020	0.020	0.000	0.000	1.00	0.075	0.075	0.000	0.000	1.00	0.060	0.060	0.000	0.000	1.00
HC	0.856	0.854	0.459	0.459	0.99	0.516	0.486	0.024	0.024	0.94	0.322	0.289	0.017	−0.007	0.67
HC_f	0.884	0.852	0.477	0.477	1.00	0.520	0.486	0.037	0.024	0.94	–	–	–	–	–
e-conn.	0.857	0.857	0.485	0.485	0.99	0.516	0.487	0.024	0.024	0.94	0.323	0.292	0.016	−0.007	0.67
tri.	0.840	0.838	0.415	0.415	0.95	0.513	0.485	0.030	0.019	0.89	0.309	0.300	0.015	0.006	0.49
2-test	0.887	0.887	0.424	0.424	0.95	0.507	0.459	0.014	0.014	0.83	0.280	0.215	0.005	−0.011	0.53
Link	0.842	0.827	0.487	0.487	1.00	0.490	0.446	0.031	0.024	1.00	–	–	–	–	–
Leiden	0.924	0.534	0.600	0.392	1.00	0.542	0.285	**0.154**	**0.152**	1.00	**0.363**	0.192	**0.142**	0.074	1.00
Leiden, γ=1	0.534	0.534	0.392	0.392	1.00	0.285	0.285	0.152	**0.152**	1.00	0.192	0.192	0.074	0.074	1.00
L-CPM	0.929	0.561	**0.617**	0.385	1.00	0.532	0.237	0.150	0.150	1.00	0.337	0.252	0.113	0.104	1.00
Infomap	**0.934**	0.890	0.577	**0.576**	1.00	**0.547**	0.505	0.041	0.030	1.00	0.356	**0.338**	0.128	**0.128**	1.00
Infomap, Markov time 1	0.921	**0.921**	0.488	0.488	1.00	**0.547**	**0.547**	0.013	0.013	1.00	0.338	**0.338**	0.128	**0.128**	1.00
LPA	0.820	0.820	0.373	0.373	1.00	0.508	0.508	0.010	0.010	1.00	0.216	0.216	0.007	0.007	1.00

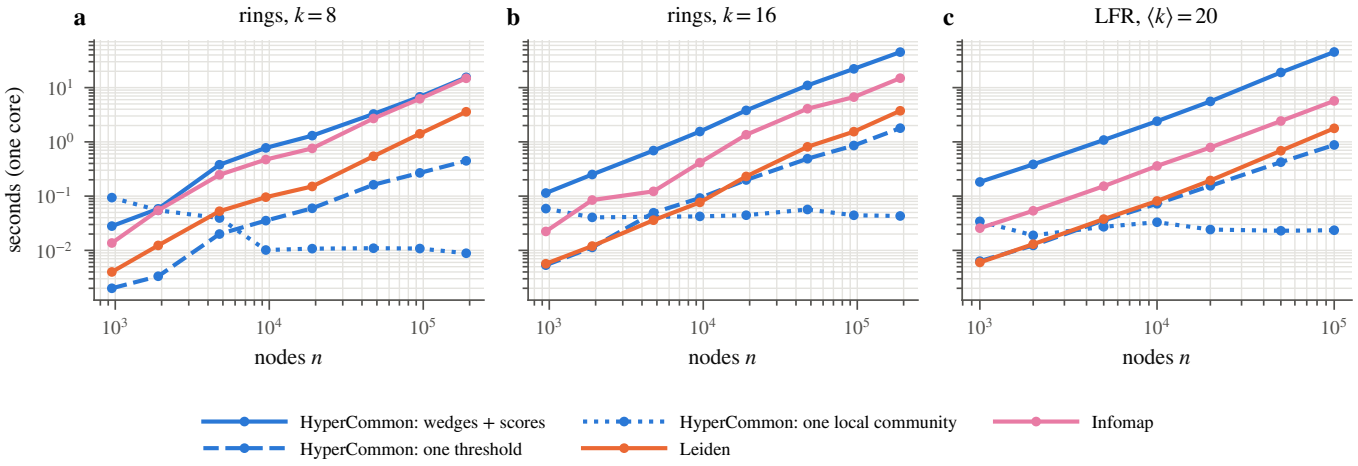


FIG. 5. Running time against network size on one core, mean of three graphs per size: ring lattices with $k = 8$ and 16 ($o_n = 0.05$, $p = 0.2$) and LFR graphs ($\langle k \rangle = 20$, $o_n = 0.1$, $\mu = 0.3$). HyperCommon's cost is split into the enumeration and scoring of all wedges, done once, the communities at one threshold, and one local query: the community grown, by the search of Proposition 2, from the first admissible triple found through a seed node drawn uniformly at random (best of three repetitions).

with NumPy and SciPy) scales as nearly linearly as the C++ implementations of Leiden and Infomap (fitted exponents of the time against n: 1.14–1.20 for the scoring of the wedges, 1.23–1.25 for Leiden and 1.18–1.25 for Infomap), consistent with Eq. (6). Once the wedges are scored, a threshold costs less than a run of Infomap at every size and less than a run of Leiden above about 2×10^4 nodes: on rings of 190 000 nodes with $k = 16$ the one-off scoring takes 45 s, after which each threshold costs 1.8 s, against 3.7 s for Leiden and 15 s for Infomap. A single cover is therefore more expensive with HyperCommon than with Leiden; the advantage is specific to sweeps, which every method needs in order to be tuned or to be chosen without labels. The local query takes less

TABLE IV. Detection on fourteen further networks with recorded groups (Netzschleuder repository [91]; sizes in Table VII), scored by best-match F1 on the labelled nodes, to which detected communities are restricted; each method at its best parameter (best) and at the parameter of largest overlapping modularity (free); stochastic methods: mean over five seeds. HC: HyperCommon with Jaccard; HC_f: the seven commonality functions together, each on 100 quantile levels of its own triple scores (in brackets the function chosen; J when Jaccard ties), the free choice taken over all of them; with labels it chooses among 700 (function, threshold) pairs per network, so its best-parameter column is an upper bound for the family, not one method's parameter grid (with 30 quantile levels per function its four wins are unchanged). "Best other": the most accurate of the twelve other methods under that protocol. Bold: best of HC, HC_f and the other methods under each protocol. Functions: J, Jaccard; cos, cosine; $I/\min$ and $I/\max$, overlap normalised by the smaller or larger neighbourhood; I, overlap count; RA, resource allocation. College football: the conference labels of Ref. [1] are those of the 2001 season while the games are from the 2000 season; corrected labels are given in Ref. [57]. Every method on every network is in Tables S25–S29.

	best parameter			free (label-free)		
Network	HC	HC_f	best other	HC	HC_f	best other
College football	0.909	**0.914** (RA)	0.893 (SCAN)	0.875	0.886 ($I/\min$)	**0.893** (SCAN)
Political books	0.784	0.786 ($I/\min$)	**0.788** (L-CPM)	0.705	0.699 (cos)	**0.767** (SCAN)
Political blogs	**0.636**	**0.636** (J)	**0.636** (Link)	0.195	0.211 (cos)	**0.571** (SLPA)
Primary school	0.405	0.826 ($I/\min$)	**0.893** (Leiden)	0.405	0.826 ($I/\min$)	**0.893** (L-CPM)
Facebook ego 0	0.399	**0.442** (I^2/U)	0.419 (SCAN)	0.390	0.378 (cos)	**0.410** (SCAN-J)
Facebook ego 107	0.552	0.616 (RA)	**0.708** (SLPA)	0.552	0.522 (cos)	**0.697** (SLPA)
Facebook ego 348	0.651	0.661 ($I/\min$)	**0.668** (SLPA)	0.479	0.479 (J)	**0.644** (SLPA)
Facebook ego 414	0.733	0.758 ($I/\min$)	**0.760** (SCAN)	0.680	**0.757** ($I/\min$)	**0.757** (L-CPM)
Facebook ego 686	0.578	**0.674** (I^2/U)	0.633 (ANGEL)	0.556	0.556 (J)	**0.620** (DEMON)
Facebook ego 698	0.826	**0.866** (I^2/U)	0.849 (CPM)	0.826	0.831 ($I/\min$)	**0.849** (CPM)
Facebook ego 1684	0.612	0.624 ($I/\min$)	**0.645** (Infomap)	0.605	0.619 (cos)	**0.645** (Infomap)
Facebook ego 1912	0.463	0.484 (cos)	**0.532** (Leiden)	0.369	0.400 ($I/\max$)	**0.525** (Leiden)
Facebook ego 3437	0.564	0.586 (cos)	**0.698** (Leiden)	0.526	0.551 (cos)	**0.644** (Leiden)
Facebook ego 3980	0.681	0.692 (cos)	**0.734** (LPA)	0.567	0.605 (cos)	**0.734** (LPA)

TABLE V. As Table IV, scored by Ω on the labelled nodes.

	best parameter			free (label-free)		
Network	HC	HC_f	best other	HC	HC_f	best other
College football	0.919	**0.927** (RA)	0.897 (Leiden)	0.836	**0.889** ($I/\min$)	0.852 (SCAN)
Political books	0.687	**0.709** (cos)	0.693 (SLPA)	0.659	**0.680** (cos)	0.676 (SLPA)
Political blogs	0.041	0.053 (cos)	**0.552** (Leiden)	0.041	0.053 (cos)	**0.540** (SLPA)
Primary school	0.060	0.713 ($I/\min$)	**0.812** (Leiden)	0.060	0.713 ($I/\min$)	**0.812** (L-CPM)
Facebook ego 0	0.286	0.339 ($I/\min$)	**0.342** (Infomap)	0.251	0.258 (cos)	**0.326** (Infomap)
Facebook ego 107	0.513	0.542 (cos)	**0.844** (L-CPM)	0.513	0.526 (cos)	**0.736** (LPA)
Facebook ego 348	0.161	0.184 (I)	**0.291** (DEMON)	−0.082	−0.082 (J)	**0.142** (DEMON)
Facebook ego 414	0.861	0.861 (J)	**0.872** (L-CPM)	**0.861**	0.810 ($I/\min$)	0.853 (SCAN)
Facebook ego 686	0.107	**0.163** (RA)	0.110 (Link)	0.075	0.075 (J)	**0.093** (SCAN-J)
Facebook ego 698	0.588	**0.712** ($I/\min$)	0.687 (Link)	0.588	0.621 ($I/\min$)	**0.647** (Leiden)
Facebook ego 1684	0.744	0.785 ($I/\min$)	**0.814** (Infomap)	0.744	0.767 (cos)	**0.814** (Infomap)
Facebook ego 1912	0.576	0.590 ($I/\max$)	**0.668** (L-CPM)	0.574	0.590 ($I/\max$)	**0.657** (Infomap)
Facebook ego 3437	0.462	0.486 (I^2/U)	**0.504** (Walktrap)	0.462	0.450 (cos)	**0.504** (Walktrap)
Facebook ego 3980	0.190	0.190 (J)	**0.535** (LPA)	0.043	0.130 (cos)	**0.535** (LPA)

than 0.1 s on average at every size, from 10^3 to 1.9×10^5 nodes.

On the full Amazon and DBLP graphs (3.3×10^5 and 3.2×10^5 nodes), timed alone on one core of the same computer (`bench/retime_big.py`), HyperCommon's scoring and its covers at all 30 thresholds take 71 and 68 s, of which 38 and 40 s are the scoring, with a peak of 3.3 and 6.3 GB of memory; Leiden's 19 resolutions take 22 and 27 minutes and Infomap's 13 Markov times 4.3 and 3.4 minutes. The overlapping modularity of the 30 covers, which the label-free choice needs, adds 9.0 and 7.7 minutes in our Python implementation; the modularity of the partitions, which the same choice needs for Leiden and Infomap, is in neither of their times. On the full YouTube graph (1.1×10^6 nodes, 1.5×10^9 wedges), the degree-ratio pruning of Sec. V E, exact for every threshold $t \geq 0.02$, discards nine wedges in ten, and the scoring and all thresholds, with the overlapping modularity

of every cover, take 10 minutes on a workstation with 28 logical processors, with a peak of 25.7 GB of memory. Leiden's resolutions took six hours and Infomap's Markov times 25 minutes on the same machine, timed without the modularity while sharing its cores with up to fifteen other jobs, so these two times are indicative only.

VIII. DISCUSSION

HyperCommon reads communities off local evidence only: a pair's shared neighbourhood, required of all three pairs of a wedge and propagated through shared pairs, never compared with a global null model. On ring lattices commonality does not depend on community size (Lemma 2), so one threshold suits rings of thirty and of six hundred nodes alike, whereas on the gradient no single resolution of either Leiden objective keeps the large rings whole and the small ones apart [59, 90] (Table S3). On the families with a few large dense rings and many small sparse ones the Constant Potts Model at its best resolution is as accurate as HyperCommon on two worlds or slightly more; HyperCommon's advantage there comes from its robustness at high noise (three big, many small) and, on two worlds, from a threshold that overlapping modularity selects better than it selects a resolution. Where HyperCommon falls short, on small sparse rings beside large dense ones, it splits them and leaves part of their nodes unassigned (Tables S10 and S12; coverage 87%–88%) rather than merging them with other communities: it reports only what local evidence supports. Degree-biased noise affects it little, for which the cap of Jaccard commonality by the degree ratio of a pair offers a mechanism: the edges that hubs gain under preferential attachment rarely pass. Overlap needs no separate membership rule: a shared node is kept in each of its communities through its own admitted edges (Corollary 1).

Theory and benchmarks meet at the exact-recovery window: with isolated merges the planted rings are recovered exactly at every threshold inside it. The ablations support each test of the triple rule (Sec. VII C): admitting open wedges, which J-thresholded 3-clique percolation does not, helps where triangles are scarce; testing the non-adjacent pair, which the two-test variant omits, keeps overlapping communities apart; and testing the wedge's two edges, which link communities [34] omit, separates HyperCommon from them most where communities overlap.

The advantage is tied to the lattice geometry rather than to unequal sizes as such (Sec. VII E). HyperCommon leads while communities are lattice-like (without labels, not on the gradient) and trails once a tenth (with labels) or three tenths (without) of each ring's edges are rewired internally, the former while the communities are still strongly clustered; it trails on all eight LFR families. Random geometric communities and rings with hubs also favour the Leiden objectives, while splitting the degree of shared nodes, as LFR graphs do, leaves its lead on the unequal rings intact. Communities joined by an admissible triple are read as one at every threshold at which that triple and the triples it touches inside them are admitted (Proposition 4 and the remark after it), the counterpart of the pair-disjointness that makes overlap clean; this limits the method where overlaps are densely connected, as in many social and collaboration networks [79, 80]. No method is best on every network [21]; local triple evidence is the right description for lattice-like communities of unequal size and density that overlap.

Five limits qualify these results. First, Ω is dominated by the largest communities: by ONMI or by recovery averaged over all communities (Tables S6 and S7), the Constant Potts Model at its best resolution is ahead on two worlds and on three big, many small, whose small rings it recovers better. Second, the label-free comparison applies one selector to every method, and for partitions it is the Newman–Girvan modularity, whose resolution limit the Constant Potts Model was introduced to avoid; it costs the Constant Potts Model more than HyperCommon on unequal rings (from 0.71 to 0.34 on three big, many small; Tables S3 and S1), so part of the label-free margin reflects the selector. At each method's own default or selection rule, HyperCommon still leads in 17 of the 28 ring settings, though the Constant Potts Model at the edge density is level with it on two worlds with overlap (Sec. VII B). Third, joining triples through shared non-adjacent pairs, which distinguishes HyperCommon's connectivity from that of link communities, adds little: with the parameter chosen per graph the edge-connected variant is within 0.02 in mean Ω on every ring setting and slightly more accurate on most LFR families; we keep the non-adjacent joins because they make every pair, not only every edge, bind at most one community (Corollary 1). Fourth, on real networks HyperCommon is competitive but rarely the most accurate (with Jaccard and its best threshold, only on college football, and tied on political blogs, by F1), its mean rank on the fourteen smaller networks is mid-field, the gains of HC_f rest on choosing the function with the labels, and the two-test variant has the higher F1 on Amazon and, with the best threshold, on nine of the fourteen smaller networks (Sec. VII F). Fifth, HyperCommon, like every method we tested, reports many communities on clustered graphs with no planted communities (Sec. VII E), so a cover it returns is not by itself evidence of community structure. Of the methods that select their own number of groups by a statistical model, the nested stochastic block model was compared: it is among the most accurate methods on LFR graphs and well behind on the ring lattices even without overlap, where other partitions do well ($\Omega = 0.50$ against 0.84 for Leiden on uniform rings with $k = 16$, and 0.28 against 0.79 for the Constant Potts Model on two worlds, both at $o_n = 0$; Table S1); BigClam and OSLOM were not compared (Sec. VI).

The threshold's best value moves with noise, so it has to be chosen per graph. Choosing it by the overlapping

modularity of the resulting cover [62] works well for HyperCommon: it retains at least 95% of the best Ω on 27 of the 36 benchmark settings and a median 94% of the best F1 on the real networks; nesting (Theorem 1) makes the covers at all thresholds one hierarchy, read off a single scoring of the wedges, so the choice costs only the evaluation of the overlapping modularity of each cover (Sec. VII G). The criterion is global, and on the gradient it selects too fine a cover; choosing thresholds separately on the branches of the hierarchy, for instance by the cluster-stability criterion of HDBSCAN [97], is the natural refinement. Degree-ratio pruning, exact for every threshold $t \geq 0.02$, keeps the sweep moderate even with hubs: on YouTube, with 1.1×10^6 nodes, every threshold of the grid is available in ten minutes on a workstation with 28 logical processors.

The commonality function is the second degree of freedom. Within degree strata every member of the family carries the signal, but the most accurate one varies from network to network, sometimes by more than the choice of method changes the accuracy. On the synthetic benchmarks Jaccard is the best single choice or close to it, and unnormalised scores fall well behind where communities differ in degree. Learning the function and the threshold from the structure of a given network is the natural next step.

DATA AND CODE AVAILABILITY

The implementation (including the vectorised code with degree-ratio pruning used for the timings), the benchmark generators, the script behind every number, table and figure, the raw results (one row per graph, method and parameter) and the per-setting paired bootstrap intervals are in release `pre-submission` of https://github.com/hypercommon/hypercommon, under the MIT licence; its `paper/README.md` lists the command that regenerates each table and figure, together with the software versions, the seeds and the hardware and load of every timing. Table S30 of the Supplemental Material lists the source of every number quoted in the text that appears in no table. The real networks are public: SNAP (https://snap.stanford.edu/data) and the Netzschleuder repository [91].

ACKNOWLEDGMENTS

D.D. acknowledges the support of the Friedrich and Ira Sachs scholarship at Bar-Ilan University.

Appendix A: Data

Table VI lists the four networks with recorded memberships, and Table VII the fourteen Netzschleuder networks of Table IV. Groups of fewer than three nodes are discarded; the SNAP top-5000 lists repeat some node sets, and Ω is scored against the distinct sets.

TABLE VI. Networks with recorded memberships. Groups: the lines of the SNAP files `all.cmty` (Amazon: `all.dedup.cmty`), as used in Sec. III; detection is scored against the top-5000 lists, which repeat some node sets (1 517 distinct sets for Amazon, 4 961 for DBLP); Ω is scored against the distinct sets. On YouTube detection is scored against every group of at least three nodes (8 385 groups on 47 135 nodes). f_{lab}: fraction of nodes with at least one recorded membership.

Network	Nodes	Edges	Groups	f_{lab}
Karate club	34	78	2	1.00
DBLP	317 080	1 049 866	13 477	0.82
Amazon	334 863	925 872	75 149	0.95
YouTube	1 134 890	2 987 624	16 386	0.046

TABLE VII. The fourteen networks of Table IV: nodes, edges, recorded groups, and the fraction of nodes that carry at least one group; groups of fewer than three nodes are discarded.

Network	Nodes	Edges	Groups	f_{lab}
College football	115	613	12	1.00
Political books	105	441	3	1.00
Political blogs	1490	16715	2	1.00
Primary school	242	8317	10	0.96
Facebook ego 0	333	2519	15	0.81
Facebook ego 107	1034	26749	9	0.46
Facebook ego 348	224	3192	14	0.97
Facebook ego 414	150	1693	7	0.89
Facebook ego 686	168	1656	14	1.00
Facebook ego 698	61	270	8	0.84
Facebook ego 1684	786	14024	15	0.97
Facebook ego 1912	747	30025	39	0.95
Facebook ego 3437	534	4813	18	0.18
Facebook ego 3980	52	146	5	0.77

Appendix B: Benchmark design, grids and settings

Table VIII lists the ring families, Table IX the parameter grids and Table X the shared nodes and fused rings of the ring benchmarks.

Ring generator. Rings are generated independently. The two nodes of each merged pair are drawn uniformly among all nodes and redrawn when they lie on the same ring or either has already been merged; the absorbed node's edges are moved to the survivor, so the graph has $n_0 - \lfloor o_n n_0 \rfloor$ nodes, $\lfloor o_n n_0 \rfloor$ of them in two rings. A rewired edge keeps one endpoint, chosen at random, and moves

the other to a non-neighbour w chosen either uniformly or preferentially, with probability proportional to $(d_w+1)^\alpha$ under the current degrees ($\alpha=1$, linear; $\alpha=2$, superlinear, which grows hubs); as $p\to 1$ the planted structure is erased. The geometry families rewire inside each ring before the merges and the noise, moving one endpoint of each chosen edge to a uniformly random non-neighbour in the same ring; a control family rewires by double-edge swaps inside each ring instead, which keeps every node's degree. In the hub families each node draws its own even degree k_i from a power law $P(k)\propto k^{-2.5}$ on $\{k_r,\dots,8k_r\}$ and is joined to its $k_i/2$ nearest neighbours on either side, so that communities keep their lattice neighbourhoods but contain hubs. In the split-overlap families each of the two merged nodes first drops a random half of its edges. Each random geometric community is a random geometric graph on the unit torus with the size and mean degree of a ring, clustered (about 0.58) but not a lattice.

LFR graphs. The reference implementation [48] (`unweighted_undirected`, compiled unmodified) with average degree $\langle k\rangle=20$, maximum degree $k_{\max}=50$, degree and community-size exponents $\tau_1=2$ and $\tau_2=1$, and community sizes between 20 and 100. The three slices at $o_n=0.1$ use $\langle k\rangle=10$, $o_m=4$, and community sizes between 50 and 250, respectively; the two clustered families, also at $o_n=0.1$, use the generator's clustering option `-C` ($C_{\mathrm{LFR}}=0.3$, 0.5).

Methods, software and settings. Implementations are those of CDLIB [98] (version 0.4.0), LEIDENALG [99] (0.11.0, with python-igraph 1.0.0), INFOMAP [100] (2.10.1) and GRAPH-TOOL [88] (3.9), except SCAN and link communities, which we implemented to expose the similarity and the cut; the stochastic block model was run under Linux on the same graphs, checked edge for edge. We also use networkx 3.6.1, numpy 2.4.1 and scipy 1.17.1. Leiden: iterated to convergence (`n_iterations`$=-1$), three restarts per resolution on the benchmarks and the fourteen smaller networks, the restart kept being the one of highest objective value under every protocol, so that no label enters the choice of restart, and one run per resolution on Amazon, DBLP and YouTube; under the native protocol the L-CPM resolution equals the edge density, at which the Constant Potts Model compares a group with an Erdős–Rényi graph of the same density [61]. Infomap: two-level, one trial. Link communities: single linkage on closed-neighbourhood Jaccard, cut at every value of HyperCommon's threshold grid, communities of fewer than three nodes discarded; under the native protocol, cut where the partition density is largest over the same grid, the criterion of Ref. [34]. SCAN: cosine similarity on closed neighbourhoods (SCAN-J: the same algorithm with Jaccard similarity); border nodes reachable from several clusters are kept in all of them. Walktrap: walks of length 4, dendrogram cut at maximum modularity (CDLIB's default). Stochastic block model: run under Ubuntu (WSL), `minimize_nested_blockmodel_dl` with default settings, seeded from the graph seed (environment in `paper/environment-sbm.yml`). Stochastic methods are seeded from the graph seed on the benchmarks, use five seeds on the fourteen smaller networks (means reported) and one on Amazon, DBLP and YouTube.

Appendix C: Proofs

This appendix proves the results of Sec. V and gives the details of the ring-lattice windows, of the fusion of rings through shared pairs and of the cost estimate.

1. Determinism, locality and nesting

Proof of Proposition 1. $\mathcal{T}_t$ is defined pointwise from G and t, the adjacency of L_t pointwise from $\mathcal{T}_t$, and the connected components of a graph are unique. □

Proof of Proposition 2. Two triples are adjacent in L_t iff they share a pair $\{x,y\}$. If $\{x,y,w\}$ is a wedge its centre is x, y or w, and in each case w is adjacent to x or y. Every neighbour of a visited triple is therefore generated from one of its three pairs, so the search traverses the component and nothing else. Admissibility of $\{x,y,w\}$ depends on $N[x],N[y],N[w]$ only. □

Proof of Theorem 1. $J\geq t_2$ implies $J\geq t_1$ and the wedge condition does not involve t, so $\mathcal{T}_{t_2}\subseteq\mathcal{T}_{t_1}$. Adjacency (sharing two nodes) does not involve t either, so L_{t_2} is induced. A connected subgraph of L_{t_2} is connected in L_{t_1}, so each component of L_{t_2}, and the union of its triples, lies inside a single component of L_{t_1}. □

2. The two ends of the hierarchy

Proof of Lemma 1. $\pi_{\mathcal{C}}$ is concentrated on c, so $\Omega_e=\pi_{\mathcal{G}}(c)<1$; a pair agrees exactly when $c_{\mathcal{G}}(i,j)=c$, so $\Omega_u=\pi_{\mathcal{G}}(c)$ as well. □

Proof of Proposition 3. (i) At $t=0$ every wedge is admissible. Two edges sharing an endpoint form a wedge, and the wedges built on a common edge share its endpoints, so a walk in the line graph of G is a walk in L_0. The line graph of a connected graph is connected, and every node of a connected graph with $|V|\geq 3$ lies on a wedge. The single community V gives $c_{\mathcal{C}}\equiv 1$, and Lemma 1 applies. (ii) $J(u,v)=1$ iff $N[u]=N[v]$, an equivalence relation whose classes are cliques; a triple is admissible at $t=1$ iff it lies within one class, and triples within one class are connected in L_1. Without classes of size at least three, $c_{\mathcal{C}}\equiv 0$ and Lemma 1 applies. For the final claim: the cover changes only at the finitely many distinct triple scores, so $\Omega(t)$ takes finitely many values on $[0,1]$ and its maximum $\Omega^\star$ is attained; under the stated hypotheses $\Omega(0)=\Omega(1)=0$ by (i) and (ii), so since $\Omega^\star\geq\Omega(t)>0$ every maximiser lies in $(0,1)$. □

TABLE VIII. Ring-lattice families: rings of size s and degree k, written $(s,k)\times$count; $n_0 = 2000$ nodes before the merges, $o_n = 0.05$ unless stated. All six unequal families were also run at $o_n = 0$ and 0.10. Geometry families: uniform $(50,8)\times 40$, two worlds, three big, many small, and the gradient, each with a fraction $q \in \{0, 0.1, 0.3, 0.6, 1\}$ of every ring's edges rewired inside the ring (one endpoint moved), noise $p \in \{0, 0.1, \ldots, 0.6\}$; the degree-preserving control uses double-edge swaps on uniform $(50,8)\times 40$ and three big, many small at $q \in \{0.1, 0.3\}$. Isolated-merge families: uniform $(50,8)\times 40$ and $(100,16)\times 20$ at $o_n = 0.01$, merges more than $2k$ apart on every ring, $p \in \{0, 0.05, 0.1, 0.2\}$. Hubs: $(50,k_i)\times 40$ with $k_i \in \{4,\ldots,32\}$, $(100,k_i)\times 20$ with $k_i \in \{8,\ldots,64\}$, and two worlds with $k_i \in \{16,\ldots,128\}$ on the five large rings and $\{4,\ldots,32\}$ on the twenty small ones, $P(k_i) \propto k_i^{-2.5}$ (mean degrees 9.2, 20.6 and 26.5). Split overlap: two worlds and three big, many small at $o_n \in \{0.05, 0.10\}$. Random geometric communities: the sizes and mean degrees of uniform $(50,8)\times 40$ and of two worlds. These control families use $p \in \{0, 0.1, \ldots, 0.8\}$. Null graphs: random geometric (2000 nodes, mean degree 8) and Holme–Kim (2000 nodes, $m = 4$, triangle probability 0.9), $p = 0$.

Family	Rings
uniform	$(50,8)\times 40$ or $(100,16)\times 20$; $o_n \in \{0, 0.05, 0.10\}$
two worlds	$(200,32)\times 5$, $(50,8)\times 20$
three big, many small	$(400,32)\times 3$, $(50,8)\times 16$
gradient	$(600,16)$, $(400,12)$, $(300,12)$, $(200,8)$, $(150,8)$, $(120,8)$, $(100,8)$, $(60,8)$, $(40,8)$, $(30,8)$
equal k, unequal s	$(100,16)\times 10$, $(50,16)\times 20$ $(100,8)\times 10$, $(50,8)\times 20$
equal s, unequal k	$(100,k)\times 5$, $k \in \{32, 24, 16, 8\}$
preferential	uniform $(50,8)\times 40$ and two worlds, $\alpha \in \{1, 2\}$

TABLE IX. Parameter grids (Sec. VI; the grids of clique percolation, SLPA, DEMON, ANGEL and SCAN are the widened ones). Logarithmically spaced grids list their end points and number of values. On Amazon, DBLP and YouTube, Leiden's RB resolution runs over $\{0.01, 0.02, 0.05, 0.1, 0.25, 0.5, 1, 2, 4, 8, \ldots, 4096\}$ (19 values, doubling from 1), its Constant Potts Model resolution over 20 values from 10^{-5} to 0.5, Infomap over the same Markov times and the HyperCommon variants over the same t grid; on Amazon and DBLP, link communities also use the t grid, and each commonality function 50 quantile levels of its own triple scores. On the fourteen smaller networks the HyperCommon variants and link communities search 100 quantile levels and the other methods the grids below; on the benchmarks each commonality function searches 29 levels $(0, 0.04, \ldots, 0.96$ and $0.97, 0.98, 0.99, 0.995)$.

Method	Grid
HyperCommon, ablations, link communities	$t \in \{0.02, 0.04, \ldots, 0.60\}$ (30)
Leiden, RB modularity	γ from 1/16 to 32 (30, logarithmic), best of 3 restarts
Leiden, Constant Potts Model	γ from 0.002 to 0.5 (30, logarithmic), best of 3 restarts
Infomap	Markov time from 0.25 to 16 (13, logarithmic)
CPM	clique size $\in \{3,\ldots,7\}$
SLPA	$r \in \{0.01, 0.02, 0.05, 0.1, 0.2, 0.3, 0.45\}$, 100 iterations
DEMON	$\epsilon \in \{0.1, 0.25, 0.5, 0.75, 0.9, 1\}$
ANGEL	threshold $\in \{0.1, 0.25, 0.5, 0.75, 0.9, 1\}$
SCAN (cosine)	$\epsilon \in \{0.1,\ldots,0.8\}$, $\mu_{\mathrm{SCAN}} \in \{2, 3, 5\}$
SCAN (Jaccard)	$\epsilon \in \{0.1,\ldots,0.6\}$, $\mu_{\mathrm{SCAN}} \in \{2, 3, 5\}$
Walktrap	walks of length 4, dendrogram cut at maximum modularity
label propagation	parameter-free
stochastic block model	level of the nested hierarchy (finest first; 1–6 levels with more than one group)

3. Ring lattices

Proof of Lemma 2. $N[x]$ is an arc of $k+1$ consecutive ring nodes, and two arcs at offset $d \le k$ overlap in $k+1-d$ nodes when $s \ge 2k+2$, which gives $J(d)$. A merged node's closed neighbourhood is the union of its two arcs, $2k+1$ nodes, and the arc of x meets only the one in its own ring, which gives $J_u(d)$. Nodes on different rings can share only merged nodes, and isolation leaves at most one, u, in both neighbourhoods. □

Proof of Theorem 2. No merges. J decreases with ring distance, and any three distinct ring nodes include a pair at distance at least two, so no triple is admissible when $t > J(2)$ and every ring is lost. For $t \le J(2)$ the consec-

TABLE X. Shared pairs in the ring benchmarks without noise (mean over ten graphs): merges, ring pairs sharing at least one node, ring pairs sharing two merged nodes within ring distance k on both rings (Proposition 4), the share of rings in such a pair, and the share of rings that exact Hyper-Common fuses with another ring just above the lower edge $1/(2k_{\min}+1)$ of the recovery window (a detected community holding at least three unshared nodes of each of two rings; analysis/double_bonds.py).

Family	o_n	merges	sharing	pair	rings	fused
uniform $(50, 8)$	0.05	100	94.2	0.8	0.04	0.11
uniform $(50, 8)$	0.10	200	180.0	2.2	0.11	0.38
uniform $(100, 16)$	0.05	100	77.2	2.6	0.24	0.61
uniform $(100, 16)$	0.10	200	126.9	8.7	0.61	1.00
two worlds	0.05	100	74.7	3.8	0.22	0.35
three big, many small	0.05	100	51.7	3.8	0.25	0.40
gradient	0.05	100	27.5	1.2	0.20	0.37
equal k, unequal s (k=16)	0.05	100	86.8	2.8	0.17	0.42
equal k, unequal s (k=8)	0.05	100	88.7	0.3	0.02	0.06
equal s, unequal k	0.05	100	79.3	3.0	0.25	0.54
two worlds	0.10	200	117.6	11.7	0.43	0.75
three big, many small	0.10	200	74.6	8.8	0.47	0.81
gradient	0.10	200	34.3	5.0	0.53	0.75
equal k, unequal s (k=8)	0.10	200	153.1	2.5	0.16	0.42
equal k, unequal s (k=16)	0.10	200	154.6	9.6	0.50	0.95
equal s, unequal k	0.10	200	124.0	12.2	0.68	0.98

utive triples $\{i, i+1, i+2\}$, with pair distances $1, 1, 2$, are admissible and each shares a pair with the next, so they form a single component covering the ring. Every other admissible wedge lies in a cyclic interval of length at most k, and moving its outer nodes inward preserves admissibility and connects it to a consecutive triple, so there are no further components. No wedge spans two rings, which share no edge, so the components are exactly the rings.

Lower edge. A wedge with nodes on two rings needs a node adjacent to both, i.e. a merged node u; its weakest pair is $J(z, y') = 1/(2k+1)$ by Eq. (3), the other two being of the form $J_u(d)$ with $d \le k/2$, which is larger. If $t \le 1/(2k+1)$, the triple $\{z, u, y'\}$ is admissible and, since $J_u(1)$ and $J(2)$ exceed $1/(2k+1)$ for $k \ge 4$, so are triples of both rings through the pairs $\{z, u\}$ and $\{u, y'\}$: the two rings fall into one component. If $t > 1/(2k+1)$ no admissible triple contains nodes of two rings other than u, so each component lies within one ground-truth community.

Upper edge. Above the lower edge every admissible triple lies within one ring, and each wedge lies in an arc of length at most k, which by isolation contains at most one merge position. Fix a merged node u at position 0 of one of its rings. Every admissible triple through u on this ring has two u-pairs, at distinct distances or on opposite sides of u, so the best is $T_u = \{-1, 0, 1\}$, with weakest pairs $J_u(1) = k/(2k+2)$: for $t > J_u(1)$, u is in no triple of the ring and is missing from its community. For $J_u(2) < t \le J_u(1)$, T_u is the only admissible triple through u on this ring; the corresponding triple on u's other ring shares only u with it and cannot attach it, so T_u connects to the ring only through its pair $\{-1, 1\}$. Every completion of that pair other than u has score at most $J(3) = (k-2)/(k+4)$, attained by ± 2. Hence for $\max(J(3), J_u(2)) < t \le J_u(1)$, T_u is an isolated component and $\{-1, 0, 1\}$ appears as a spurious community.

Conversely, let $t \le J_u(1)$ and $t \le \max(J(3), J_u(2))$. The consecutive triples that avoid u are admissible (their pairs are at distances 1, 1, 2, and $J(2) > J_u(1) \ge t$ for $k \ge 4$), and so is T_u. If $t \le J(3)$, the bridges $\{-2, -1, 1\}$ and $\{-1, 1, 2\}$, whose pairs avoid u and lie at distances 1, 2 and 3, are admissible; they share $\{-2, -1\}$ and $\{1, 2\}$ with the consecutive triples on either side of the merge and $\{-1, 1\}$ with T_u. If instead $t \le J_u(2)$, the consecutive triples $\{-2, -1, 0\}$ and $\{0, 1, 2\}$ through u are admissible and share $\{-1, 0\}$ and $\{0, 1\}$ with T_u. Either way the triples listed so far form one component per ring. Every other admissible triple T of the ring joins it through *contraction moves*: replace an outer node of T whose next node towards the other two (skipping u if $u \notin T$) is not already in T by that node, and stop when the three nodes are consecutive in the ring order (with u removed if $u \notin T$). A move keeps two nodes of T, so successive triples share a pair; it increases no ring distance within the triple, so the triple remains a wedge; and it does not decrease the score, because pairs not involving u have commonality $J(d)$ and pairs involving u have $J_u(d)$, both decreasing in d, and a move that replaces u itself turns each u-pair at distance d into a pair at distance $d' \le d$ with $J(d') \ge J(d) > J_u(d)$. The moves therefore end, through admissible triples, at a consecutive triple, a bridge, or a triple through u already listed, so all admissible triples within each ring are connected. Since $J_u(1) < J(2)$ for $k \ge 4$, the no-merge condition is implied, and Eq. (4) follows. □

Isolation. The proof uses isolation only through the facts that an arc of $k+1$ consecutive nodes contains at most one merge position and that nodes of different rings share at most one merged neighbour. For a ring pair merged twice the separation $k+1$ is sharp: shared nodes k apart on both rings raise the lower edge to $1/k$.

Unequal degrees. For one isolated merge between rings of even degrees $k_a, k_b \ge 4$ and sizes at least $2k_a + 2$ and $2k_b + 2$, the same argument applies with the commonalities of Lemma 2 for unequal degrees. The cross pair has $J = 1/(k_a + k_b + 1)$, which gives the lower edge of Eq. (5). On the ring of degree x, with y the degree of the other ring, the upper-edge argument runs with $J_u(1) = x/(x+y+2)$, $J_u(2) = (x-1)/(x+y+3)$ and $J(3) = (x-2)/(x+4)$, which gives $U(x, y)$; both rings must be recovered, which gives the minimum over $U(k_a, k_b)$ and $U(k_b, k_a)$.

Obstruction for unequal degrees. A shared node between a dense and a sparse community is held more weakly by the sparse one, so the window of such a merge narrows and moves down, with upper edge $k_b/(k_a+k_b+2)$ for $k_a \gg k_b$. It cannot overlap the window of merges between two sparse rings, whose lower edge is $1/(2k_b+1)$,

once $k_a \geq 2k_b^2 - 2$: beyond that degree contrast no single threshold recovers a family that also contains merges between two sparse rings, even without noise.

4. Shared pairs and pair-disjointness

Proof of Proposition 4. If $T_A \neq T_B$, the two triples share exactly the two nodes u_1, u_2 and are adjacent in L_t. □

Fusion of rings. Connectivity through shared pairs is what lets a community grow through its own open wedges, and Proposition 4 is its counterpart. On ring lattices, three communities that pairwise share single nodes are joined when two of the three shared-node pairs are adjacent and the third lies within distance k; for $k = 8$ and pairwise adjacent shared nodes two apart, the rings are recovered only for $4/13 < t \leq 4/9$, and for $k = 4$ they are never recovered. Two rings joined only through two shared merged nodes, with no other merge within ring distance k of them, can be fused above the lower edge $1/(2k+1)$ only when those nodes lie within ring distance k of each other on both rings, or within $k/2$ on one ring and within $3k/2$ on the other; for $k = 8$, shared nodes 2 apart on one ring and 11 apart on the other fuse the rings for $t \leq 3/23$. Theorem 2's isolation hypothesis excludes both cases. Whether fusion then occurs depends on the threshold, since the merge lowers the commonality of the shared pair: two rings with $s = 50$, $k = 8$ that share two nodes four positions apart are fused at small t but recovered separately at $t = 0.4$, where the shared pair, with $J = 1/3$, is not admitted.

Proof of Corollary 1. Two distinct admissible triples that contain the same pair share exactly two nodes and are therefore adjacent in L_t, so all admissible triples through the pair lie in one component. □

5. Cost

Every commonality needed is available from the wedges themselves: for a pair $\{a, b\}$, $|N[a] \cap N[b]|$ is the number of common neighbours of a and b, that is of centred wedges a–c–b (a triangle contributes once to each of its three pairs), plus 2 if $a \sim b$, so one pass over the centred wedges, with pairs indexed by hashing, counts all of them. Our implementation finds components on the incidence graph between admitted triples and their pairs over a grid of thresholds. Alternatively, as in sequential clique percolation [45], a single union–find sweep over the triples in decreasing order of score builds the merge tree of the triple components at all thresholds in $O(W \log W)$, from which the cover at any one threshold is read in $O(W)$ more. For heavy-tailed degree distributions the hubs dominate through W, as for every triangle-based method.

Degree-ratio pruning. In the pruning of Sec. V E, every wedge through an edge with $\rho < t_{\min}$ is discarded. Common neighbours reached through discarded edges must still be counted, for example by intersecting the adjacency lists of the kept pairs; counting only the kept wedges would underestimate J. This is the size filter of exact set-similarity joins [101, 102], applied to closed neighbourhoods of sizes $d_u + 1$ and $d_v + 1$. The pruning applies to every rule that tests the edges of a wedge, including our three ablations, but not to link communities, which score a wedge by its non-adjacent pair alone, whose commonality is not bounded by the degrees of the wedge's edges, so every wedge through a hub has to be enumerated. Our implementation counts the common neighbours of the kept pairs from the full adjacency matrix, and pruned and unpruned runs give identical covers on its test graphs, which include a hub joined to many leaves, and on the full Amazon and DBLP graphs at all 30 thresholds $t \geq 0.02$ of our grid.

[1] M. Girvan and M. E. J. Newman, Community structure in social and biological networks, Proceedings of the National Academy of Sciences **99**, 7821 (2002).

[2] M. E. J. Newman, *Networks*, 2nd ed. (Oxford University Press, 2018).

[3] S. Fortunato, Community detection in graphs, Physics Reports **486**, 75 (2010).

[4] S. Fortunato and M. E. J. Newman, 20 years of network community detection, Nature Physics **18**, 848 (2022).

[5] J. Xie, S. Kelley, and B. K. Szymanski, Overlapping community detection in networks: The state-of-the-art and comparative study, ACM Computing Surveys **45**, 43 (2013).

[6] S. Fortunato and D. Hric, Community detection in networks: A user guide, Physics Reports **659**, 1 (2016).

[7] M. E. J. Newman and M. Girvan, Finding and evaluating community structure in networks, Physical Review E **69**, 026113 (2004).

[8] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, Fast unfolding of communities in large networks, Journal of Statistical Mechanics: Theory and Experiment **2008**, P10008 (2008).

[9] V. A. Traag, L. Waltman, and N. J. van Eck, From Louvain to Leiden: guaranteeing well-connected communities, Scientific Reports **9**, 5233 (2019).

[10] M. Rosvall and C. T. Bergstrom, Maps of random walks on complex networks reveal community structure, Proceedings of the National Academy of Sciences **105**, 1118 (2008).

[11] T. P. Peixoto, Hierarchical block structures and high-resolution model selection in large networks, Physical Review X **4**, 011047 (2014).

[12] T. P. Peixoto, Model selection and hypothesis testing for large-scale network models with overlapping groups, Physical Review X **5**, 011033 (2015).

[13] B. Ball, B. Karrer, and M. E. J. Newman, Efficient and principled method for detecting communities in networks, Physical Review E **84**, 036103 (2011).

[14] J. Yang and J. Leskovec, Overlapping community detection at scale: A nonnegative matrix factorization approach, in *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining* (2013) pp. 587–596.

[15] A. Lancichinetti, F. Radicchi, J. J. Ramasco, and S. Fortunato, Finding statistically significant communities in networks, PLoS ONE **6**, e18961 (2011).

[16] U. N. Raghavan, R. Albert, and S. Kumara, Near linear time algorithm to detect community structures in large-scale networks, Physical Review E **76**, 036106 (2007).

[17] S. Gregory, Finding overlapping communities in networks by label propagation, New Journal of Physics **12**, 103018 (2010).

[18] J. Xie, B. K. Szymanski, and X. Liu, SLPA: Uncovering overlapping communities in social networks via a speaker-listener interaction dynamic process, in *2011 IEEE 11th International Conference on Data Mining Workshops* (2011) pp. 344–349.

[19] A. Clauset, Finding local community structure in networks, Physical Review E **72**, 026132 (2005).

[20] A. Lancichinetti, S. Fortunato, and J. Kertész, Detecting the overlapping and hierarchical community structure in complex networks, New Journal of Physics **11**, 033015 (2009).

[21] L. Peel, D. B. Larremore, and A. Clauset, The ground truth about metadata and community detection in networks, Science Advances **3**, e1602548 (2017).

[22] A. Ghasemian, H. Hosseinmardi, and A. Clauset, Evaluating overfit and underfit in models of network community structure, IEEE Transactions on Knowledge and Data Engineering **32**, 1722 (2020).

[23] M. S. Granovetter, The strength of weak ties, American Journal of Sociology **78**, 1360 (1973).

[24] G. Kossinets and D. J. Watts, Empirical analysis of an evolving social network, Science **311**, 88 (2006).

[25] D. J. Watts and S. H. Strogatz, Collective dynamics of 'small-world' networks, Nature **393**, 440 (1998).

[26] T. P. Peixoto, Disentangling homophily, community structure, and triadic closure in networks, Physical Review X **12**, 011004 (2022).

[27] P. Jaccard, The distribution of the flora in the alpine zone, New Phytologist **11**, 37 (1912).

[28] E. A. Leicht, P. Holme, and M. E. J. Newman, Vertex similarity in networks, Physical Review E **73**, 026120 (2006).

[29] T. Zhou, L. Lü, and Y.-C. Zhang, Predicting missing links via local information, The European Physical Journal B **71**, 623 (2009).

[30] L. Lü and T. Zhou, Link prediction in complex networks: A survey, Physica A: Statistical Mechanics and its Applications **390**, 1150 (2011).

[31] B. Barzel and A.-L. Barabási, Network link prediction by global silencing of indirect correlations, Nature Biotechnology **31**, 720 (2013).

[32] S. Feizi, D. Marbach, M. Médard, and M. Kellis, Network deconvolution as a general method to distinguish direct dependencies in networks, Nature Biotechnology **31**, 726 (2013).

[33] X. Xu, N. Yuruk, Z. Feng, and T. A. J. Schweiger, SCAN: A structural clustering algorithm for networks, in *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2007) pp. 824–833.

[34] Y.-Y. Ahn, J. P. Bagrow, and S. Lehmann, Link communities reveal multiscale complexity in networks, Nature **466**, 761 (2010).

[35] T. S. Evans and R. Lambiotte, Line graphs, link partitions, and overlapping communities, Physical Review E **80**, 016105 (2009).

[36] G. Palla, I. Derényi, I. Farkas, and T. Vicsek, Uncovering the overlapping community structure of complex networks in nature and society, Nature **435**, 814 (2005).

[37] I. Derényi, G. Palla, and T. Vicsek, Clique percolation in random networks, Physical Review Letters **94**, 160202 (2005).

[38] I. J. Farkas, D. Ábel, G. Palla, and T. Vicsek, Weighted network modules, New Journal of Physics **9**, 180 (2007).

[39] J. Cohen, *Trusses: Cohesive subgraphs for social network analysis*, Tech. Rep. (National Security Agency, 2008).

[40] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, Querying k-truss community in large and dynamic graphs, in *Proceedings of the 2014 ACM SIGMOD In-*

ternational Conference on Management of Data (2014) pp. 1311–1322.
[41] A. R. Benson, D. F. Gleich, and J. Leskovec, Higher-order organization of complex networks, Science **353**, 163 (2016).
[42] F. Battiston, G. Cencetti, I. Iacopini, V. Latora, M. Lucas, A. Patania, J.-G. Young, and G. Petri, Networks beyond pairwise interactions: Structure and dynamics, Physics Reports **874**, 1 (2020).
[43] K. Kovalenko, I. Sendiña-Nadal, N. Khalil, A. Dainiak, D. Musatov, A. M. Raigorodskii, K. Alfaro-Bittner, B. Barzel, and S. Boccaletti, Growing scale-free simplices, Communications Physics **4**, 43 (2021).
[44] P. S. Chodrow, N. Veldt, and A. R. Benson, Generative hypergraph clustering: From blockmodels to modularity, Science Advances **7**, eabh1303 (2021).
[45] J. M. Kumpula, M. Kivelä, K. Kaski, and J. Saramäki, Sequential algorithm for fast clique percolation, Physical Review E **78**, 026109 (2008).
[46] Z. Zhang, L. Cui, Z. Pan, A. Fang, and H. Zhang, A triad percolation method for detecting communities in social networks, Data Science Journal **17**, 30 (2018).
[47] A. Lancichinetti, S. Fortunato, and F. Radicchi, Benchmark graphs for testing community detection algorithms, Physical Review E **78**, 046110 (2008).
[48] A. Lancichinetti and S. Fortunato, Benchmarks for testing community detection algorithms on directed and weighted graphs with overlapping communities, Physical Review E **80**, 016118 (2009).
[49] F. Radicchi, C. Castellano, F. Cecconi, V. Loreto, and D. Parisi, Defining and identifying communities in networks, Proceedings of the National Academy of Sciences **101**, 2658 (2004).
[50] R. A. Jarvis and E. A. Patrick, Clustering using a similarity measure based on shared near neighbors, IEEE Transactions on Computers **C-22**, 1025 (1973).
[51] J.-P. Onnela, J. Saramäki, J. Hyvönen, G. Szabó, D. Lazer, K. Kaski, J. Kertész, and A.-L. Barabási, Structure and tie strengths in mobile communication networks, Proceedings of the National Academy of Sciences **104**, 7332 (2007).
[52] A. Ghasemian, H. Hosseinmardi, A. Galstyan, E. M. Airoldi, and A. Clauset, Stacking models for nearly optimal link prediction in complex networks, Proceedings of the National Academy of Sciences **117**, 23393 (2020).
[53] C. E. Tsourakakis, J. Pachocki, and M. Mitzenmacher, Scalable motif-aware graph clustering, in *Proceedings of the 26th International Conference on World Wide Web* (2017) pp. 1451–1460.
[54] T. Chen and C. E. Tsourakakis, Parallel motif-based community detection, in *2024 IEEE International Conference on Big Data* (2024) pp. 509–518.
[55] A. E. Sarıyüce, C. Seshadhri, A. Pınar, and Ü. V. Çatalyürek, Finding the hierarchy of dense subgraphs using nucleus decompositions, in *Proceedings of the 24th International Conference on World Wide Web* (2015) pp. 927–937.
[56] Q. F. Lotito, F. Musciotto, A. Montresor, and F. Battiston, Hyperlink communities in higher-order networks, Journal of Complex Networks **12**, cnae013 (2024).
[57] T. S. Evans, Clique graphs and overlapping communities, Journal of Statistical Mechanics: Theory and Experiment **2010**, P12037 (2010).
[58] S. G. Aksoy, C. Joslyn, C. Ortiz Marrero, B. Praggastis, and E. Purvine, Hypernetwork science via high-order hypergraph walks, EPJ Data Science **9**, 16 (2020).
[59] S. Fortunato and M. Barthélemy, Resolution limit in community detection, Proceedings of the National Academy of Sciences **104**, 36 (2007).
[60] P. Ronhovde and Z. Nussinov, Local resolution-limit-free Potts model for community detection, Physical Review E **81**, 046114 (2010).
[61] V. A. Traag, P. Van Dooren, and Y. Nesterov, Narrow scope for resolution-limit-free community detection, Physical Review E **84**, 016114 (2011).
[62] H. Shen, X. Cheng, K. Cai, and M.-B. Hu, Detect overlapping and hierarchical community structure in networks, Physica A: Statistical Mechanics and its Applications **388**, 1706 (2009).
[63] B. Tóth, T. Vicsek, and G. Palla, Overlapping modularity at the critical point of k-clique percolation, Journal of Statistical Physics **151**, 689 (2013).
[64] M. Coscia, G. Rossetti, F. Giannotti, and D. Pedreschi, DEMON: a local-first discovery method for overlapping communities, in *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2012) pp. 615–623.
[65] G. Rossetti, ANGEL: efficient, and effective, node-centric community discovery in static and dynamic networks, Applied Network Science **5**, 26 (2020).
[66] P. Pons and M. Latapy, Computing communities in large networks using random walks, Journal of Graph Algorithms and Applications **10**, 191 (2006).
[67] M. T. Schaub, R. Lambiotte, and M. Barahona, Encoding dynamics for multiscale community detection: Markov time sweeping for the map equation, Physical Review E **86**, 026112 (2012).
[68] M. Kheirkhahzadeh, A. Lancichinetti, and M. Rosvall, Efficient community detection of network flows for varying Markov times and bipartite networks, Physical Review E **93**, 032309 (2016).
[69] A. Epasto, S. Lattanzi, and R. Paes Leme, Ego-splitting framework: From non-overlapping to overlapping clusters, in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2017) pp. 145–154.
[70] D. Deinega, *Overlapping Communities Detection in Complex Networks*, Master's thesis, Ariel University (2023).
[71] W. W. Zachary, An information flow model for conflict and fission in small groups, Journal of Anthropological Research **33**, 452 (1977).
[72] J. Yang and J. Leskovec, Defining and evaluating network communities based on ground-truth, Knowledge and Information Systems **42**, 181 (2015).
[73] C. Elkan and K. Noto, Learning classifiers from only positive and unlabeled data, in *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2008) pp. 213–220.
[74] D. Hric, R. K. Darst, and S. Fortunato, Community detection in networks: Structural communities versus ground truth, Physical Review E **90**, 062805 (2014).
[75] See Supplemental Material at [URL will be inserted by publisher] for the full benchmark tables under the four protocols; ONMI, per-community recovery, coverage and numbers of communities; the paired intervals behind the scorecard; the selector and grid-edge diag-

nostics; the control families and null graphs; the commonality family on the benchmarks; every method on the real networks; and the sources of numbers quoted in the text.

[76] G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval* (McGraw-Hill, 1983).

[77] E. Ravasz, A. L. Somera, D. A. Mongru, Z. N. Oltvai, and A.-L. Barabási, Hierarchical organization of modularity in metabolic networks, Science **297**, 1551 (2002).

[78] L. M. Collins and C. W. Dent, Omega: A general formulation of the Rand index of cluster recovery suitable for non-disjoint solutions, Multivariate Behavioral Research **23**, 231 (1988).

[79] J. Yang and J. Leskovec, Structure and overlaps of ground-truth communities in networks, ACM Transactions on Intelligent Systems and Technology **5**, 26 (2014).

[80] J. Yang and J. Leskovec, Community-affiliation graph model for overlapping network community detection, in *2012 IEEE 12th International Conference on Data Mining* (2012) pp. 1170–1175.

[81] J. Yang and J. Leskovec, Overlapping communities explain core–periphery organization of networks, Proceedings of the IEEE **102**, 1892 (2014).

[82] C. I. Del Genio, T. Gross, and K. E. Bassler, All scale-free networks are sparse, Physical Review Letters **107**, 178701 (2011).

[83] A. D. Broido and A. Clauset, Scale-free networks are rare, Nature Communications **10**, 1017 (2019).

[84] I. Voitalov, P. van der Hoorn, R. van der Hofstad, and D. Krioukov, Scale-free networks well done, Physical Review Research **1**, 033034 (2019).

[85] I. Kanovsky, Small world models for social network algorithms testing, Procedia Computer Science **1**, 2341 (2010).

[86] P. Holme and B. J. Kim, Growing scale-free networks with tunable clustering, Physical Review E **65**, 026107 (2002).

[87] J. Reichardt and S. Bornholdt, Statistical mechanics of community detection, Physical Review E **74**, 016110 (2006).

[88] T. P. Peixoto, The graph-tool python library, figshare (2014).

[89] A. F. McDaid, D. Greene, and N. Hurley, Normalized mutual information to evaluate overlapping community finding algorithms (2011), arXiv:1110.2515.

[90] A. Lancichinetti and S. Fortunato, Limits of modularity maximization in community detection, Physical Review E **84**, 066122 (2011).

[91] T. P. Peixoto, The Netzschleuder network catalogue and repository (2020), https://networks.skewed.de.

[92] J. McAuley and J. Leskovec, Learning to discover social circles in ego networks, in *Advances in Neural Information Processing Systems 25* (2012) pp. 539–547.

[93] V. Krebs, Books about US politics, unpublished network of Amazon.com co-purchases, http://www.orgnet.com/.

[94] M. E. J. Newman, Modularity and community structure in networks, Proceedings of the National Academy of Sciences **103**, 8577 (2006).

[95] L. A. Adamic and N. Glance, The political blogosphere and the 2004 U.S. election: divided they blog, in *Proceedings of the 3rd International Workshop on Link Discovery* (2005) pp. 36–43.

[96] J. Stehlé, N. Voirin, A. Barrat, C. Cattuto, L. Isella, J.-F. Pinton, M. Quaggiotto, W. Van den Broeck, C. Régis, B. Lina, and P. Vanhems, High-resolution measurements of face-to-face contact patterns in a primary school, PLoS ONE **6**, e23176 (2011).

[97] R. J. G. B. Campello, D. Moulavi, and J. Sander, Density-based clustering based on hierarchical density estimates, in *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, Lecture Notes in Computer Science, Vol. 7819 (2013) pp. 160–172.

[98] G. Rossetti, L. Milli, and R. Cazabet, CDLIB: a python library to extract, compare and evaluate communities from complex networks, Applied Network Science **4**, 52 (2019).

[99] V. A. Traag, leidenalg: Implementation of the Leiden algorithm in C++ for Python, version 0.11.0, https://github.com/vtraag/leidenalg.

[100] D. Edler, A. Holmgren, and M. Rosvall, The MapEquation software package, infomap version 2.10.1, https://mapequation.org.

[101] A. Arasu, V. Ganti, and R. Kaushik, Efficient exact set-similarity joins, in *Proceedings of the 32nd International Conference on Very Large Data Bases* (2006) pp. 918–929.

[102] R. J. Bayardo, Y. Ma, and R. Srikant, Scaling up all pairs similarity search, in *Proceedings of the 16th International Conference on World Wide Web* (2007) pp. 131–140.